



Advances and challenges in learning from experience replay

Daniel Eugênio Neves¹ · Lucila Ishitani¹ ·
Zenilton Kleber Gonçalves do Patrocínio Júnior¹

Accepted: 30 November 2024 / Published online: 20 December 2024
© The Author(s) 2024

Abstract

From the first theoretical propositions in the 1950s to its application in real-world problems, Reinforcement Learning (RL) is still a fascinating and complex class of machine learning algorithms with overgrowing literature in recent years. In this work, we present an extensive and structured literature review and discuss how the Experience Replay (ER) technique has been fundamental in making various RL methods in most relevant problems and different domains more data efficient. ER is the central focus of this review. One of its main contributions is a taxonomy that organizes the many research works and the different RL methods that use ER. Here, the focus is on how RL methods improve and apply ER strategies, demonstrating their specificities and contributions while having ER as a prominent component. Another relevant contribution is the organization in a facet-oriented way, allowing different perspectives of reading, whether based on the fundamental problems of RL, focusing on algorithmic strategies and architectural decisions, or with a view to different applications of RL with ER. Moreover, we start by presenting a detailed formal theoretical foundation of RL and some of the most relevant algorithms and bring from the recent literature some of the main trends, challenges, and advances focused on ER formal basement and how to improve its propositions to make it even more efficient in different methods and domains. Lastly, we discuss challenges and open problems and present relevant paths to feature works.

Keywords Reinforcement learning · Experience replay · Deep reinforcement learning · Survey

✉ Daniel Eugênio Neves
daniel.eugenio@sga.pucminas.br

Lucila Ishitani
lucila@pucminas.br

Zenilton Kleber Gonçalves do Patrocínio Júnior
zenilton@pucminas.br

¹ Instituto de Ciências Exatas e Informática, Pontifícia Universidade Católica de Minas Gerais, Avenida Dom José Gaspar, 500, Belo Horizonte, Brazil30535-901, Minas Gerais

1 Introduction

Machine learning is an artificial intelligence research field in which mathematical and statistical models applied through computational algorithms seek to provide machines with intelligent behavior and the ability to learn from their experiences. In particular, Reinforcement Learning (RL) research is dedicated to developing intelligent computational entities called agents, which try to learn an action policy to interact with the environment and perform a given task, i.e., each agent action on an arbitrary environment state results in a new state and a reward signal and the objective is to learn an optimal policy that maximizes the total expected reward in the long run. If the agent learns an optimal action policy, it will have learned to perform the task. Based on the formal framework of the Markov Decision Process (MDP), different RL methods have been proposed from their early formulations in the 1950s to deal with problems involving decision-making sequences. These processes run by defining value functions to measure the agent choices, either using dynamic programming in tabular environments to find exact solutions for discrete action values or approximating these value functions in high-dimensional state spaces with discrete or continuous action values.

Temporal Difference Learning (TD-learning) was an essential formulation for developing trial-and-error-based learning methods. It allows the agent to adjust itself from a target value based on the reward estimate on a resultant state in the previous interaction, making it possible for many recent algorithms to converge to an optimal policy even if acting sub-optimally, as long as it keeps updating its value function. Moreover, this was the basis for a whole class of off-policy and model-free methods. To cope with some efficiency issues inherent to TD-learning and make TD-based approaches more data efficient, Lin (1992) proposed a fundamental technique called Experience Replay (ER), consisting of reusing previous agent experiences (i.e., previous state transitions) in updating its value function by storing and uniformly sampling them from a replay buffer. This strategy brought unique benefits related to using artificial neural networks in approximating value functions because it decorrelates the agent training data.

A wide range of modern methods employ ER. At the same time, many authors seek to improve it by investigating how to make better use of the replay buffer, how to sample better experiences for agent learning, how to deal with a size-limited buffer (and how big it should be), how to model that buffer and many other questions. Due to its benefits for data-efficient reinforcement learning, research on the use and improvement of ER increased exponentially from 2016, with a disproportionate volume of publications compared to previous years. Despite its relevance and the growing number of publications and methods that use it in some way, we have not found a paper in the last two years dedicated to a literature review specifically about the evolution and application of ER techniques. Zhu et al. (2024) present a survey about multi-agent deep reinforcement learning with communication (Comm-MADRL) focusing on agents' communication processes. Hickling et al. (2023) present a review of methods and applications to explainability in deep reinforcement learning. Shen and Zhao (2024) review the task construction settings and the application of RL for various natural language processing problems. Like these, other review works present evaluations or applications of classes of RL methods (e.g., in Peng et al. (2024), Elharrouss et al. (2024), Mishra and Arora (2024)), but do not have ER as the central focus in an extensive review. McKenzie and McDonnell (2022) present a review focused on the progression of value-

based Reinforcement Learning in the last five years until its publication. They highlight the various and diverse algorithm changes in different aspects, including ER, which figures among many other factors, techniques, and strategies discussed in the review. Despite this, the authors emphasize advances in the recurrent experience replay in the distributed reinforcement learning algorithm.

Therefore, this work aims to review from the early bases until current reinforcement learning methods to formally understand and compare how they use ER and how it makes them more data efficient. Moreover, we seek to contribute to understanding its fundamental ideas and highlight its many theoretical and empirical open problems still being investigated to organize and point out possible future works and research directions. In this way, we are narrowly interested in those works that propose changes or new methods using ER and especially interested in those works that investigate the ER itself and delve into its theoretical issues and empirical investigations. One of the main contributions of this work is a taxonomy that organizes the many research studies and their different methods. It focuses on how they improve and apply experience replay strategies, evidentiating their specificities and contributions, and having the ER as the central topic. Another relevant contribution is how we organize knowledge in a facet-oriented way, allowing different perspectives of reading, whether based on the fundamental problems of RL, focusing on algorithmic strategies and architectural decisions, or with a view to different applications.

This work is organized as follows. In Sect. 2, we present and discuss the theoretical background and some related work. Section 3 presents the Experience Replay foundations. Section 4 presents relevant deep reinforcement learning methods that focus on how they use ER and the differences in their propositions to explore agents' experiences. Section 5 discusses some of the main research challenges and trends. Section 6 discusses recent research in Experience Replay and some direction for future works. Section 7 presents a structured summary of the research works, methods, and challenges discussed in this extensive review. Finally, we draw some conclusions in Sect. 8.

2 Background

Reinforcement Learning uses the formal framework of the Markov Decision Process (MDP) to define the interaction between a learning agent and its environment regarding states, actions, and rewards. An MDP is defined by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, R, \gamma)$, so that: $\mathcal{S}$ represents a set of states; $\mathcal{A}$ is a set of actions $\mathcal{A} = \{a_1, a_2, \dots, a_n\}$; $\mathcal{P}(s' | s, a)$ is the probability of transiting from state s to s' ($s, s' \in \mathcal{S}$) by taking action $a \in \mathcal{A}$; $\mathcal{R}$ is a reward function mapping each state-action pair to a reward in $\mathbb{R}$; and $\gamma \in [0, 1]$ is a discount factor. A policy π represents the agent's behavior, and the value $\pi(a | s)$ represents the probability of taking action a in state s . At each time step t , the agent observes a state $s_t \in \mathcal{S}$ and chooses an action $a_t \in \mathcal{A}$ that determines the reward $r_t = \mathcal{R}(s_t, a_t)$ and the next state $s_{t+1} \sim \mathcal{P}(\cdot | s_t, a_t)$, causing a transition of states $T(s, a, s')$. A discounted sum of future rewards is called return $R_t = \sum_{t'=t}^{\infty} \gamma^{t'-t} r_{t'}$. The agent aims to learn (or approximate) an optimal policy π^* that maximizes the expected long-term (discounted) reward value. These processes imply non-deterministic search problems and stochastic decision sequences for selecting actions from observing each state of the environment resulting from a previous decision. In this way, each agent's action determines the immediate reward and, more importantly, influences subse-

quent environment states and future rewards. While the immediate reward informs about the result of an action performed in the current state, the long-term expected reward allows evaluation of the action policy by a value function.

Bellman formulated the MDPs as a stochastic version of the optimal control problem and described two value functions using the concept of states of a dynamical system. The state-value function $v_\pi(s)$ seeks to estimate the expected total (discounted) reward value when the agent starts in state s and follows the policy π . Describing $v_\pi(s)$, one can note (see Eqs. 1–4) the expected sum of future rewards for the states reached by adopting the policy π and performing the sequences of state transitions. In Eq. (5), it is clear the dynamic nature of the computation of $v_\pi(s)$, and in Eq. (6) we have the discount value γ on the total expected reward.

$$v_\pi(s) = \mathbb{E}_\pi[r_1 + r_2 + \dots + r_T \mid s_t = s] \quad (1)$$

$$= \mathbb{E}_\pi[r_t] + \mathbb{E}_\pi[r_{t+1} + r_{t+2} + \dots + r_T \mid s_t = s] \quad (2)$$

$$= \sum_a \pi(s, a) R(s, a) + \mathbb{E}_\pi[r_{t+1} + r_{t+2} + \dots + r_T \mid s_t = s] \quad (3)$$

$$= \sum_a \pi(s, a) R(s, a) + \sum_a \pi(s, a) \sum_{s'} T(s, a, s') \mathbb{E}_\pi[r_{t+1} + \dots + r_T \mid s_t = s'] \quad (4)$$

$$= \sum_a \pi(s, a) R(s, a) + \sum_a \pi(s, a) \sum_{s'} T(s, a, s') v_\pi(s') \quad (5)$$

$$= \sum_a \pi(s, a) \left[R(s, a) + \gamma \sum_{s'} T(s, a, s') v_\pi(s') \right] \quad (6)$$

In the form presented by Sutton and Barto (2018) in Eq. 7, the state-value function $v_\pi(s)$ demonstrates the notions of probability and transition, making the relationship explicit between the value of a state and the values of its successor states. In turn, the action-value functions $q_\pi(s, a)$ seek to estimate the total expected reward if the agent takes an action a in the state s and follows the policy π , allowing it to assess the utility of each possible action at that state (Eq. 8).

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')] \quad (7)$$

$$q_\pi(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') \left[\sum_a \pi(s', a') q_\pi(s', a') \right] \quad (8)$$

In MDPs, a policy π is better or equivalent to another policy π' if $v_\pi(s) \geq v_{\pi'}(s), \forall s \in S$. In all cases, at least one optimal policy π^* is better than or equal to all others. These policies share the same optimal state-value function $v^*(s) = \max_\pi v_\pi(s)$, which is the high-

est value that can be obtained for each state, and the same optimal action-value function $q^*(s, a) = \max_a q_\pi(s, a)$, $\forall s, a \in S, A$ (Sutton and Barto 2018). It is possible to write the optimal action-value function relative to the optimal state-value function so that $q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma v^*(S_{t+1}) \mid S_t = s, A_t = a]$. Since $v^*(s)$ is an optimal state-value function, Bellman's equation demonstrates that the value of a state under an optimal policy must be equal to the expected return for the best action in that state:

$$\begin{aligned} v^*(s) &= \max_{a \in A(s)} q_\pi^*(s, a) \\ &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v^*(s')] \end{aligned} \quad (9)$$

In turn, the Bellman optimality equation for the action-value function can be defined as follows:

$$\begin{aligned} q^*(s, a) &= \mathbb{E}[R_{t+1} + \gamma \max_{a'} q^*(S_{t+1}, a') \mid S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r \mid s, a) [r + \gamma \max_{a'} q^*(s', a')] \end{aligned} \quad (10)$$

From v^* , one can find π^* and vice versa, both of which are solutions for MDPs. For each state, one or more actions will produce the maximum value in Bellman's equation, and any policy that maximizes v^* will be optimal. While knowledge of the optimal state-value function v^* makes it possible to search for the optimal policy π^* , knowing the optimal action-value function q^* makes it easy to choose optimal actions. This way, for any state s , the agent only needs to find one action that maximizes the value of q^* because it effectively stores the results for all searches one step ahead. It gives the optimal expected return as a local and immediately available value for each state-action pair. It allows one to select optimal actions without knowing the possible successor states with their respective values or, in other words, the dynamics of the environment.

Bellman's equations allow for finding optimal policies. Still, they are rarely used in practice, as they demand exhaustive searches in the space of states and actions besides assuming that the dynamics of the environment are precisely known, which is not always true. It imposes limitations on a class of methods known as Dynamic Programming (DP) (Szepesvári 2010), which can converge to optimal policies with exact solutions and provide the basis for understanding several other reinforcement learning methods since many of them consist of attempts to achieve the same results but at a lower computational cost, and without the need for a perfect model of the environment (Sutton and Barto 2018). The main idea of DP is to use value functions to search for optimal policies, assuming that the environment is described as an MDP, the sets of states, actions, and rewards are finite, and there is a probability function that describes the environment's dynamics. In this way, DP can compute the value functions, transforming the Bellman equations into updating rules. In this sense, there are four main related algorithms: policy evaluation, policy improvement, policy iteration, and value iteration.

The policy evaluation method is an iterative solution that uses the state-value function. For a sequence of functions $\{v_0, \dots, v_k\}$ mapping the states to values, the value for v_0 is arbitrarily chosen and updated from the values computed in subsequent iterations using the Bellman equation for v_π as an update rule, in which the state-value function at iteration

$v_{k+1}(s)$ considers the expected discounted return obtained for the next possible state s' in the previous iteration, for every state $s \in S$. It is possible to demonstrate that the sequence of value-functions v_k converges to v_π when $k \rightarrow \infty$. At each iteration, to produce v_{k+1} from v_k , the algorithm applies the same operation to each state s , assigning a new value obtained from the previous values of the state s' , a successor of s , and the expected immediate reward for each possible transition under the policy in evaluation. In this way, each iteration updates the value of each state to produce a new approximation of the state-value function v_{k+1} :

$$\begin{aligned} v_{k+1}(s) &= \mathbb{E}_\pi[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s] \\ &= \sum_a \pi(s | a) \sum_{s', a} p(s', r | s, a) [r + \gamma v_\pi(s')] \end{aligned} \quad (11)$$

Even having determined the state-value function v_π for a policy π , it is still possible to check whether or not it would be better to select a given action a different from what determined the policy in that state. One way to answer this question is to compute the result for the action-value function $q_\pi(s, a)$, introducing a policy improvement step into the policy evaluation method:

$$\begin{aligned} q_\pi(s, a) &= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s, A_t = a] \\ &= \sum_{s', a} p(s', r | s, a) [r + \gamma v_\pi(s')] \end{aligned} \quad (12)$$

The policy will change if $q_\pi(s, a) > v_\pi(s)$, as it will be better to choose the action a in the state s and then follow the policy π instead of following π all the time. So, it is expected that it will be better to select the action a every time the state s is found and that this new policy will be the best overall. Therefore, it is possible to consider changes in all states for all possible actions in a greedy strategy, selecting in each state the best action according to $q_\pi(s, a)$, so that the new policy π' is given by:

$$\begin{aligned} \pi'(s) &= \operatorname{argmax}_a q_\pi(s, a) \\ &= \operatorname{argmax}_a \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s, A_t = a] \\ &= \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')] \end{aligned} \quad (13)$$

This is a special case of the policy improvement theorem. Let two policies be π and π' so that, for all $s \in S$, $q_\pi(s, \pi'(s)) \geq v_\pi(s)$, the policy π' must be as good as or better than π (i.e. π' must have an expected reward greater than or equal to π), where $v_{\pi'}(s) \geq v_\pi(s)$. This result applies particularly to the original policy π and the modified policy π' . If $q_\pi(s, a) > v_\pi(s)$, then the modified policy will be better than the original policy. Given a policy and its value function, it is possible to evaluate a policy change in a single state for a given action (Sutton and Barto 2018). Once a policy π has been improved, a policy iteration process produces a sequence of improvements until it reaches an optimal policy π^* and an optimal value function v^* , as each new action policy is guaranteed to be better than the previous one unless this is already the optimal policy. Considering that a finite MDP has a

finite number of policies, this process must converge to an optimal value function and policy in a finite number of iterations.

Although convergence to the optimal policy and the optimal value function is guaranteed, each policy iteration step includes the policy evaluation, which is also iterative, leading to a computation that requires many scans in the space of states. However, truncating the policy evaluation step is possible without losing the policy iteration convergence guarantee. A special case occurs when the policy evaluation stops after a single scan (i.e., after an update step for each state). This method is called value iteration and is a simple update process that combines policy improvement and short policy evaluation steps, as in Eq. 14, for all $s \in S$:

$$\begin{aligned} v_{k+1}(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')] \end{aligned} \quad (14)$$

It achieves faster convergence by interposing multiple policy evaluation scans between each policy improvement scan; meanwhile, its output consists of a deterministic policy $\pi \approx \pi^*$ such that $\pi(s) = \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) [r + \gamma v(s')]$.

According to Sutton and Barto (2018), Temporal Difference Learning (TD) is one of the most relevant ideas of reinforcement learning. It allows learning to occur directly from the agent experience without the need for a model of the dynamics of the environment. It can update estimates based on other learned estimations before reaching a final state, which is a clear advantage over DP methods concerning computational efficiency. There are variations of the TD method regarding the number of steps applied to calculate the temporal difference, called by the acronym $TD(\lambda)$, where λ is the number of steps. $TD(0)$ is a particular case and updates the estimate of $v(s_t)$ for an iteration t using the observed immediate reward r and the estimate of $v(s_{t+1})$ at iteration $t + 1$. Thus, it waits only for the next iteration to form a target value $r + \gamma v(s_{t+1})$ and updates the value of s_t immediately after the transition to s_{t+1} , so that $v(s_t) \leftarrow v(s_t) + \alpha [r + \gamma v(s_{t+1}) - v(s_t)]$, where α is a learning rate, γ is the discount value, and s_t and s_{t+1} are respectively the environment's states at iterations t and $t + 1$. The difference $\gamma v(s_{t+1}) - v(s_t)$ is known as the Temporal Difference Error (TDE). Sampling-based updatings, like those used in TD methods, are distinct from those used in dynamic programming methods, as they are based on a single successor state and not on a complete probability distribution over all possible successors. Thus, TD methods are independent of a model of the environment, are naturally implemented online and incrementally, and converge to an optimal policy.

Q-Learning (Watkins and Dayan 1992) is a model-free and off-policy algorithm that applies successive steps to update the estimates for the action-value function $Q(s, a)$ (that approximates the long-term expected discounted reward value of executing an action from a given state) using TD-learning and minimizing the TD-error (defined by the difference in Eq. 15). This function is named Q-function, and its estimated returns are known as Q-values. A higher Q-value indicates that an action a would yield better long-term results in state s . Q-Learning converges to an π^* even if it is not acting optimally every time as long as it keeps updating the Q-value estimates for all the pairs of state-action and generates a variation of the usual stochastic approximation conditions through subsequent changes of α , as we can describe in Eq. 15.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \quad (15)$$

Applying Q-learning to those problems where the space of states and actions is too large to learn all the actions' values in all possible states or when these states are multidimensional data, one can achieve a good approximate solution by learning a parameterized value function $Q(s, a, \Theta_t)$ as in Eq. 16 (Van Hasselt et al. 2016).

$$\Theta_{t+1} = \Theta_t + \alpha[r_{t+1} + \gamma \max_a Q(s_{t+1}, a, \Theta_t) - Q(s_t, a_t, \Theta_t)] \nabla_{\Theta_t} Q(s_t, a_t, \Theta_t) \quad (16)$$

One can see that the target function in the TD-error calculation consists of a greedy policy defined by the *max* function. In this way, the maximum over the estimated values is implicitly used to estimate the highest return value, which can lead to considerable maximization bias. For example, in a state s , there may be many actions for which the actual return value $q(s, a)$ is zero, but the estimated values $Q(s, a)$ may be distributed over negative and positive values. In this case, always taking the maximum introduces a clear positive bias in this set of actions. Such a maximization bias can lead the agent to choose misguided actions more often in a given state. One way to approach this problem is to use two independent estimates, $Q_1(s, a)$ and $Q_2(s, a)$, of the actual value of $q(s, a)$, $\forall a \in A$. So, we can use $Q_1(s, a)$ to determine the action $a^* = \operatorname{argmax}_a Q_1(s, a)$ and $Q_2(s, a)$ to provide the estimate $Q_2(s, a^*) = Q_2(s, \operatorname{argmax}_a Q_1(s, a))$. This way, it is also possible to perform the same process by reversing the roles of $Q_1(s, a)$ and $Q_2(s, a)$ to obtain a second estimate of reduced bias from $Q_1(s, \operatorname{argmax}_a Q_2(s, a))$. This is the approach proposed by van Hasselt (2010) to formulate the method called Double Q-Learning (Eq. 17), in which only one of the estimates is updated at each training step based on a probability value. The authors demonstrated that this approach reduced bias by decomposing the *max* operation in the target function into action selection and evaluation, which improved the update of the action-value function, making it more stable by diminishing the overestimation of the Q-values.

$$Q_1(s, a) \leftarrow Q_1(s, a) + \alpha[r + \gamma Q_2(s_{t+1}, \operatorname{argmax}_a Q_1(s_{t+1}, a)) - Q_1(s, a)] \quad (17)$$

To use parameterized value functions, Double Q-Learning learns two value functions using two different sets of weights Θ and Θ' and, at each update step, one set is used to select the action greedily and the other to evaluate it, as defined in Eqs. 18 and 19.

$$y_t = r_t + \gamma Q(s_{t+1}, \operatorname{argmax}_a Q(s_{t+1}, a, \Theta_t), \Theta'_t) \quad (18)$$

$$\Theta_{t+1} = \Theta_t + \alpha[y_t - Q(s_t, a_t, \Theta_t)] \nabla_{\Theta_t} Q(s_t, a_t, \Theta_t) \quad (19)$$

As an agent interacts with stochastic, non-deterministic, and partially observable environments, *exploration* and *exploitation* are also essential concepts, and how to balance them is a recurring challenge. Exploration refers to the experimentation of the new and the generation of new knowledge but tends to maximize risks to expand the agent's knowledge. Exploitation relates to the knowledge assimilated, the maximization of efficiency and performance, the minimization of risks, and the refinement of the knowledge already acquired. In this way, to increase the accumulated reward value, an agent seeks to select actions already experienced and which produced good results, but it is necessary to try new actions to dis-

cover new ones that may provide greater rewards, thus choosing between obtaining rewards quickly or having a chance to select better action in the future. The agent must try various actions and progressively pick the best ones. This dilemma is a complex problem and has not yet been exhausted in the literature.

Bellemare et al. (2017) discuss a distributed perspective on the return value in contrast to modeling its expectation and propose an algorithm that applies the Bellman equation to learn from approximate value distributions. Considering that the value function Q estimates the random value return (resulting from the probabilities of the transitions), the authors describe its distributional nature like in Eq. 20, where Z is the value distribution, and R (the reward function) is explicitly a random variable. A stationary policy π maps each estate to a probability distribution over the action space.

$$Z(s, a) \stackrel{D}{=} R(s, a) + \gamma Z(s', A') \quad (20)$$

The authors define the value Z^π as the sum of discounted rewards along the agent's interaction with the environment, the value functions as vectors in $\mathbb{R}^{S \times A}$, and consider the expected reward function as one of those vectors. Therefore, they define a Bellman operator τ^π and an optimality operator τ like in Eqs. 21 and 22, where P is the transition function (as defined in Sect. 2). Instead of expectation, they consider the full distribution of the variable Z^π , which they call *value distribution*. Moreover, they also discuss the theoretical behavior of the distributional analogs of the Bellman operator in the control setting.

$$\tau^\pi Q(s, a) := \mathbb{E}R(s, a) + \gamma \mathbb{E}_{P, \pi} Q(s', a') \quad (21)$$

$$\tau Q(s, a) := \mathbb{E}R(s, a) + \gamma \mathbb{E}_P \max_{a'} Q(s', a') \quad (22)$$

Finally, the authors present their state-of-the-art results of modeling and applying the distributional value within a DQN agent evaluated on ALE (Bellemare et al. 2013) and demonstrate considerable improvements in agent performance.

3 Experience replay

After an agent has performed a sequence of actions and received a return value, knowing how to assign credit (or discredit) to each state-action pair consists of a difficult problem called *Temporal Credit Assignment*. Temporal Difference Learning (TD) represents one of the main techniques to deal with this problem, despite being a slow process, especially when it involves credit propagation over a long sequence of actions. For example, Adaptive Heuristic Critic – AHC (Sutton 1992) and Q-Learning (Watkins and Dayan 1992), which represent the first TD-learning-based methods in RL, are characterized by high convergence times. An effective technique called Experience Replay (ER) was proposed by Lin (1992) to speed up the credit attribution process and consequently reduce convergence time by storing agent experiences in a replay buffer and uniformly sampling past experiences to update the agent model. One of its main motivations is that relevant algorithms become inefficient when they use trial-and-error experiences only once to adjust the evaluation functions and

then discard them. This is because some agents' experiences can be rare, while others can be expensive, such as those involving penalties. Moreover, using ER with random sampling reduces the effect of the correlation of the data (which represents the environment states) and improves its nonstationary distribution when using neural networks to approximate the value functions because it softens the distribution over many previous experiences (Mnih et al. 2013). The correlation between consecutive observations of the environment's states can lead the minor updates (on the approximate model) to generate considerable changes in the policy learned by the agent (Mnih et al. 2015). So it can change the data distribution and the relation between the action-value functions in calculating the TD-error.

Lin (1992) compared Experience Replay to two other techniques: (i) *Learning Action Models for Planning* and (ii) *Teaching*, regarding shortening the trial-and-error process. In *Learning Action Models for Planning*, the agent does not need to learn a model of actions by itself, as in such cases when there is a perfect environment's model or when it can quickly grasp a good one. In *Teaching*, lessons from a teacher (i.e., a human player or maybe another agent) demonstrate how to get from an initial to a final state and accomplish the task goal. Those lessons store selected actions, state transitions, and attained rewards. This way, an agent can repeat these lessons several times, similar to what it could do with its own experiences. The author applied the three techniques in eight reinforcement learning frameworks based on AHC and Q-learning: (i) AHCON (Connectionist AHC-learning); (ii) AHCON-R (AHCON using Experience Replay); (iii) AHCON-M (AHCON using Actions Model); (iv) AHCON-T (AHCON using Experience Replay and Teaching); (v) QCON (Connectionist Q-learning); (vi) QCON-R (QCON with Experience Replay); (vii) QCON-M (QCON with Actions Model); and (viii) QCON-T (QCON with Experience Replay and Teaching). These frameworks seek to learn a policy evaluation function approximated by neural networks and adjusted using TD and backpropagation.

According to Lin (1992), using ER in AHCON-R and QCON-R did not improve the agent performance (compared to AHCON and QCON) but speeded up the convergence in all experiments. In turn, using action models in AHCON-M and QCON-M could speed up the convergence time, but this did not happen during the experiments. When comparing AHCON-T and QCON-T with AHCON-R and QCON-R, no significant differences existed in the less complex environments. However, in the complex environments, AHCON-T and QCON-T were considerably faster. Therefore, the author states that the advantage of using *Teaching* becomes more significant as the task becomes more demanding and has demonstrated the superiority of ER over the *Actions Model* when agents need to learn a model of the environment by themselves. However, the latter method is superior if a perfect action model is provided. Nevertheless, it does not seem advantageous, especially in problems with large spaces of states and actions, nondeterministic scenarios, and nontabular solutions, where there is no way to provide a perfect action model to the agent considering all possible situations.

Experience Replay has some limitations. As it repeats experiences through uniform sampling, if those samples define policies very different from what the agent is learning, it can underestimate the evaluation and utility functions, which affects methods that use neural networks because whenever it adjusts the weights for a given state, this affects the entire model concerning many (or perhaps all) other states. Besides, the memory of experiences does not differentiate relevant experiences due to the uniform sampling and overwrites many agent experiences due to the buffer size limitation. That points to the need for more sophisticated

strategies that can emphasize experiences capable of contributing more to agent learning in the sense of what was proposed by Schaul et al. (2016). Another relevant problem is related to the size of the replay buffer. According to Mnih et al. (2015), all DQN-based methods (some of the most relevant are presented in Sect. 4) used a fixed replay memory size of 1 M transitions. Recently, some research works investigated the effects of small and large buffers (Zhang and Sutton 2017; Liu and Zou 2018). In turn, Neves et al. (2022) used a small dynamic memory to explore the replay of the experience and the dynamics of the transitions, reducing the number of experiences required for agent learning.

4 Experience replay in deep reinforcement learning

Looking at the history of reinforcement learning, some of the most recent and significant improvements have arisen from the possibility of approximating value functions from multidimensional data. At this point, adopting artificial neural networks was a promising proposition deeply investigated in the early related literature, including Lin (1992). However, the proposal of using Convolutional Neural Networks to approximate the action-value functions together with Experience Replay was a game-changing approach presented by Mnih et al. (2013). From that, and because ER reduces nonstationarity and decorrelates the agent's updates, contributing to the stabilization when using deep neural networks, the recent research in the literature has been working based on these two fundamental findings and bringing many relevant discoveries mainly in this, but not only, relevant aspects: (i) approximating value function; (ii) reducing bias; (iii) composing better value functions; (iv) improving data efficiency; and (v) dealing with continuous-valued action spaces. Therefore, this section starts the discussions regarding ER in Deep RL, supported by some fundamental works and other new studies about these relevant aspects.

4.1 Variations on convolutional neural networks in Q-learning and double Q-learning-based approaches

Deep Q-Network (DQN) (Mnih et al. 2013) and Double Deep Q-Network (DDQN) (Van Hasselt et al. 2016) are two relevant methods based on Q-Learning and Double Q-Learning with ER. These methods achieved state-of-the-art results and human-level performance in learning to play a set of Atari 2600 games emulated in the Arcade Learning Environment (ALE) (Bellemare et al. 2013; Machado et al. 2018), which allows complex challenges for RL agents such as non-determinism, stochasticity, and exploration. To approximate the action-value functions, the authors used Convolutional Neural Networks (CNN) on the representations of environment states obtained from the video game frames without any prior information regarding the games, no manually extracted features, and no knowledge regarding the internal state of the ALE emulator. Thus, agent learning occurred only from video inputs, reward signals, the set of possible actions, and the final state information of each game. The authors attributed their state-of-the-art results mainly to the ability of their CNN to represent the games' states.

Mnih et al. (2015) improved DQN by changing how CNN is used, as shown in Algorithm 1. Instead of using the same network (with the same parameters Θ) to approximate both the action-value function $Q(s, a, \Theta)$ and the target action-value function $Q(s', a, \Theta)$,

the authors used independent sets of parameters Θ and Θ' for each network. Thus, only the function $Q(s, a, \Theta)$ has its parameters Θ updated by backpropagation. The parameters Θ' are updated directly (i.e., copied) from the values of Θ with a certain frequency, remaining unchanged between two consecutive updates. Thus, only the *forward pass* is performed when the network is used with the parameters Θ' to predict the value of the target function. Specifically, at each time-step t , a transition (or experience) is defined by a tuple $\tau_t = (s_t, a_t, r_t, s_{t+1})$, in which s_t is the current state, a_t is the action taken at that state, r_t is the reward received at t , and s_{t+1} is the state resulting after taking action a_t . Recent experiences are stored to construct a replay buffer $\mathcal{D} = \{\tau_1, \tau_2, \dots, \tau_{N_D}\}$, in which N_D is the buffer size. Therefore, a CNN can be trained on samples $(s_t, a_t, r_t, s_{t+1}) \sim U(\mathcal{D})$, drawn uniformly at random from the pool of experiences by iteratively minimizing the following loss function,

$$\mathcal{L}_{DQN}(\Theta_i) = \mathbb{E}_{(s_t, a_t, r_t, s_{t+1}) \sim U(\mathcal{D})} \left[\left(r_t + \gamma \max_{a'} Q(s_{t+1}, a', \Theta') - Q(s_t, a_t, \Theta_i) \right)^2 \right], \quad (23)$$

in which Θ_i are the parameters from the i -th iteration. Instead of using the same network, another one provides the target values $Q(s_{t+1}, a', \Theta')$ used to calculate the TD-error, decoupling any feedback that may result from using the same network to generate its own targets.

The Algorithm 2 addresses the trade-off between exploration and exploitation through an ϵ -Greedy strategy which selects an action from a homogeneous distribution over the set of possible actions with a given probability (exploration); otherwise, it uses the CNN that approximates the $Q(s, a)$ function to select the action that maximizes the estimation of the Q-value (exploitation). Generally, the value of the hyperparameter ϵ decreases over time, causing the agent to explore a lot at first but gradually transiting to use more and more of the acquired knowledge.

input: batch size k , learning rate α , training frequency tf , total iterations number T , ϵ -Greedy probability ϵ , discount factor γ , replay memory capacity n , update target frequency utf

- 1 Initialize replay memory $\mathcal{D} \leftarrow \emptyset$, $t \leftarrow 0$;
- 2 Initialize Θ at random; Initialize $\Theta' = \Theta$;
- 3 Initialize $env \leftarrow EnvironmentInstance$; $s_t \leftarrow env.InitialState()$;
- 4 **for** $t \leftarrow 1$ **to** T **do**
- 5 $a_t \leftarrow \epsilon\text{-Greedy}(s_t, \epsilon, \Theta)$;
- 6 $s_{t+1}, r_t \leftarrow env.step(s_t, a_t)$;
- 7 $StoreTransition(\mathcal{D}, (s_t, a_t, r_t, s_{t+1}))$;
- 8 **if** $t \bmod tf == 0$ **then**
- 9 **for** $k \leftarrow 1$ **to** K **do**
- 10 $\tau = (s_t, a_t, r_t, s_{t+1}) \sim UniformSample(\mathcal{D})$;
- 11 $Q_{target} \leftarrow r_t + \gamma \times \max_{a'} Q(s_{t+1}, a', \Theta')$;
- 12 $Q_{value} \leftarrow Q(s_t, a_t, \Theta)$;
- 13 $\mathcal{L}(\Theta) \leftarrow (Q_{target} - Q_{value})^2$;
- 14 $\Delta \leftarrow \Delta + \mathcal{L}(\Theta) \times \nabla_{\Theta} Q_{value}$;
- 15 $\Theta \leftarrow \Theta + \alpha \times \Delta$;
- 16 **if** $t \bmod utf == 0$ **then**
- 17 $\Theta' \leftarrow \Theta$;
- 18 $\Delta \leftarrow 0$;
- 19 $s_t \leftarrow s_{t+1}$;

Algorithm 1 DQN – deep Q-networks

```

input : state  $s$ ,  $\epsilon$ -Greedy probability  $\epsilon$ , CNN parameters  $\Theta$ 
output: action  $a$ 
1 if  $\text{randomNumber} > \epsilon$  then
2   |  $q\text{Values} \leftarrow \text{CNN.Forward}(s, \Theta)$ ;
3   |  $a \leftarrow \text{argmax}(q\text{Values})$ ;
4 else
5   |  $i \leftarrow \text{random}(\#actions)$ ;
6   |  $a \leftarrow \text{actions}[i]$ 
7 return  $a$ ;

```

Algorithm 2 ϵ -Greedy

One can note a bias in DQN, such as in Q-Learning. According to Van Hasselt et al. (2016), it can lead to overestimated high action values. Still, these values would not be a problem if all action values were uniformly higher, which probably does not occur. However, it is more likely that overestimation is common during learning, mainly when action values are inaccurate. In this way, the real problem is if that overestimation is not uniform and rises more often from state-action pairs that lead to suboptimal policies. The authors showed the occurrence of overestimations in DQN and proposed the DDQN based on Double Q-Learning. Since Double Q-Learning learns two value functions using two different sets of weights Θ and Θ' , it is possible to compare Q-Learning to Double Q-Learning, rewriting its target value to untangle action selection and evaluation – Eqs. 24 and 25.

$$Y_t^Q = R_{t+1} + \gamma Q(S_{t+1}, \text{argmax}_a Q(S_{t+1}, a, \Theta_t), \Theta_t) \quad (24)$$

$$Y_t^{\text{DoubleQ}} = R_{t+1} + \gamma Q(S_{t+1}, \text{argmax}_a Q(S_{t+1}, a, \Theta_t), \Theta'_t) \quad (25)$$

The authors demonstrated that DDQN (see Algorithm 3) reduced bias using the formulation of Double Q-Learning by decomposing the $\max$ operation in the target function into action selection and action evaluation, which improved the action-value functions updates making it more stable by diminishing the overestimation of the Q-values. The target value changes from Eqs. 26, 27. The update of the target network is performed in the same way as in the DQN, periodically copying the updated weights from the online network, which approximates the evaluation function.

$$Y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a', \Theta') \quad (26)$$

$$Y_t = r_t + \gamma Q(s_{t+1}, \text{argmax}_a Q(s_{t+1}, a, \Theta), \Theta') \quad (27)$$

input: batch size k , learning rate α , training frequency tf , total iterations number T , ϵ -Greedy probability ϵ , discount factor γ , replay memory capacity n , update target frequency utf

- 1 Initialize replay memory $\mathcal{D} \leftarrow \emptyset$, $t \leftarrow 0$;
- 2 Initialize Θ at random; Initialize $\Theta' = \Theta$;
- 3 Initialize $env \leftarrow EnvironmentInstance$; $s_t \leftarrow env.InitialState()$;
- 4 **for** $t \leftarrow 1$ **to** T **do**
- 5 $a_t \leftarrow \epsilon\text{-Greedy}(s_t, \epsilon, \Theta)$;
- 6 $s_{t+1}, r_t \leftarrow env.step(s_t, a_t)$;
- 7 $StoreTransition(\mathcal{D}, (s_t, a_t, r_t, s_{t+1}))$;
- 8 **if** $t \bmod tf == 0$ **then**
- 9 **for** $k \leftarrow 1$ **to** K **do**
- 10 $\tau = (s_t, a_t, r_t, s_{t+1}) \sim UniformSampling(\mathcal{D})$;
- 11 $Q_{target} \leftarrow r_t + \gamma \times Q(s_{t+1}, \argmax_a Q(s_{t+1}, a, \Theta_t), \Theta_t)$;
- 12 $Q_{value} \leftarrow Q(s_t, a_t, \Theta)$;
- 13 $\mathcal{L}(\Theta) \leftarrow r_t + \gamma \times Q_{target} - Q_{value}$;
- 14 $\Delta \leftarrow \Delta + \mathcal{L}(\Theta) \times \nabla_{\Theta} Q_{value}$;
- 15 $\Theta \leftarrow \Theta + \alpha \times \Delta$;
- 16 **if** $t \bmod utf == 0$ **then**
- 17 $\Theta' \leftarrow \Theta$;
- 18 $\Delta \leftarrow 0$;
- 19 $s_t \leftarrow s_{t+1}$;

Algorithm 3 DDQN – double deep Q-networks

Some research works sought to investigate video frames as sequences in the input of the neural network that approximate the value action function in DQN-based methods to improve the perception of movements, speed up the trial-and-error process, and deal with problems in which agents only observe a reward signal after long sequences of decision making. For Hausknecht and Stone (2015), mapping states to actions based only on the four previous game states (stacking the frames in a pre-processing step) prevents the DQN agent from achieving the best performance in games that require remembering faraway events from a large number of frames, because in those games the future states and rewards depend on several previous states. Therefore, the authors proposed using Long Short-term Memory (LSTM) instead of the first fully connected layer, just after the series of convolutional layers in the original DQN neural network architecture, to use the little history better. The authors demonstrate a trade-off between using a non-recurrent network with a long history of observations or a recurrent network with just one frame at each iteration step. They stated that a recurrent network is a viable approach for dealing with observations from multiple states. However, it presents no systematic benefits compared to stacking these observations in the input layer of a plain CNN. Moreno-Vera (2019) proposed a similar approach using DDQN instead of DQN.

Wang et al. (2016) proposed using a new deep neural network architecture called Dueling Networks instead of classical (well-known) architectures such as CNNs, LSTMs, or MLPs to improve model-free reinforcement learning methods. This architecture can generalize learning across actions without changes to the underlying reinforcement learning algorithm.

Their architecture uses two estimators in the same network, one for the state-value function $V(s, \theta, \beta)$ and another for the so-called state-dependent action advantage function $A(s, a, \theta, \alpha)$, defined by two streams of fully connected layers (following the convolutional layers) whose outputs are the (scalar) state-value and a vector of advantages for each action. Equation 28 combines these two outputs and produces the final Q-value estimations, in which α and β are the parameters of the two streams of fully connected layers, and θ represents the parameters of the convolutional layers.

$$Q(s, a, \theta, \alpha, \beta) = V(s, \theta, \beta) + \left(A(s, a, \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a', \theta, \alpha) \right) \quad (28)$$

As the output is also Q-value estimates for each action in the input states, the dueling network architecture can replace the original neural networks in other algorithms such as DQN and DDQN, with adaptation only regarding backpropagation. The authors demonstrated improved experimental results using uniform and Prioritized Experience Replay (PER) (Schaul et al. 2016).

4.2 Dealing with continuous-valued action spaces in off-policy RL

The DQN-based methods, such as the DDQN and Dueling Networks, either with original ER or PER, achieved state-of-the-art (at the time of their respective publications) in learning to act directly from high dimensional states, interacting with nondeterministic environments in a stochastic way, and approximating the optimal policy from discrete-valued and low-dimensional action spaces. However, many interesting problems, such as physical control tasks, have continuous (real-valued) and high-dimensional action spaces ($\mathcal{A} = \mathbb{R}^N$). In these cases, for the DQN to find the action that maximizes the Q-value estimate, an iterative optimization process would be necessary at each step of the agent. Therefore, Lillicrap et al. (2016) have based on the Deterministic Policy Gradient (DPG) algorithm (Silver et al. 2014) and the DQN to propose an actor-critic and model-free algorithm called Deep Deterministic Policy Gradient (DDPG) that uses deep neural networks with ER and can learn over continuous action spaces. According to the authors, DDPG can find policies whose performance is competitive (sometimes better) with those found by a planning algorithm with full access to the dynamics of challenging physical control problems that involve complex multi-joint movements, cartesian coordinates, unstable and rich contact dynamics, and gait behavior. They have evaluated their agent in learning action policies from video-frame pixels and physical control data (such as joint angles), using the same hyperparameters and network architecture in different challenges in simulated physical environments through a physics engine originally proposed for model-based control called MuJoCo (Todorov et al. 2012).

From the derivations of the Bellman equation presented in Sect. 2 to define the action-value function in Eq. 8, one can note how the target policy could be described as a func-

tion $\mu : \mathcal{S} \rightarrow \mathcal{A}$ if this policy is deterministic, to replace the expectation in the target, as described by Lillicrap et al. (2016), changing from Eqs. 29, 30.

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1}}[r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi}[Q^\pi(s_{t+1}, a_{t+1})]] \quad (29)$$

$$Q^\mu(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1}}[r(s_t, a_t) + \gamma Q^\mu(s_{t+1}, \mu(s_{t+1}))] \quad (30)$$

As in $\mathcal{MDP}$ s, the discounted sum of future rewards R depends on the policy π ; the authors denote its distribution over the visited states as ρ^π . Therefore, it is possible to learn the function Q^μ off-policy using transitions obtained from a different stochastic behavior policy they referenced as β and a distribution ρ^β . Thus, an approximator for the Q-value function parameterized by Θ^Q could be optimized by minimizing the loss obtained in Eq. 31.

$$L(\Theta^Q) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta}[(Q(s_t, a_t \mid \Theta^Q) - r_t + \gamma Q(s_{t+1}, \mu(s_{t+1}) \mid \Theta^Q))^2] \quad (31)$$

The DPG algorithm applies a parameterized function $\mu(s \mid \theta^\mu)$ (the actor) to define the current policy by deterministically mapping states to actions and updating its parameter using the *policy gradient* (i.e., the gradient of the policy's performance) (Silver et al. 2014). This update technique applies the chain rule to the expected return from the start distribution J concerning the actor parameters, as in Eqs. 32 and 33. In turn, it learns the Q-value function $Q(s, a)$ (the critic) using the Bellman equation as in Q-Learning (Lillicrap et al. 2016).

$$\nabla_{\theta^\mu} J \approx \mathbb{E}_{s_t \sim \rho^\beta}[\nabla_{\theta^\mu} Q(s, a \mid \theta^Q) \mid s = s_t, a = \mu(s_t \mid \theta^\mu)] \quad (32)$$

$$= \mathbb{E}_{s_t \sim \rho^\beta}[\nabla_a Q(s, a \mid \theta^Q) \mid s = s_t, a = \mu(s_t) \nabla_{\theta^\mu} \mu(s \mid \theta^\mu) \mid s = s_t] \quad (33)$$

DDPG (see Algorithm 4) applies modifications to DPG to use the contribution of DQN in approximating the Q-value function from the high-dimensional states space and to use the policy gradient to deal with high-dimensional and continuous action spaces. It also uses a replay buffer with uniform sampling. One change is in the updating of the target function. Instead of copying the weights directly from the updating to the target neural network (such as in DQN), the authors create a copy of the critic and actor networks, $Q'(s, a \mid \theta^{Q'})$ and $\mu'(s \mid \theta^{\mu'})$, and use them to estimate the target values, then update these target networks by slowly (for stability) tackling the updating neural networks making $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$, with $\tau \ll 1$. This way, the authors obtained stable targets y_i to train the critic network consistently. To deal with learning from low-dimensional physical feature vector observations, whose components may have different units and scales, such as position and velocity, the authors applied the batch normalization technique (Ioffe and Szegedy 2015) to the state input and all layers of the μ network and the layers of the Q network before its action

input layer. Because it is off-policy, DDPG can deal with exploration (a difficult problem in continuous action spaces) independently of the learning algorithm. While DQN uses an ϵ -greedy approach (see Algorithm 2), Lillicrap et al. (2016) use an Ornstein-Uhlenbeck process (Uhlenbeck and Ornstein 1930) to add a noise $\mathcal{N}$ to the actor policy and generate temporally correlated exploration, where $\mu'(s_t) = \mu(s_t | \theta_t^\mu) + \mathcal{N}$.

input: batch size k , discount factor γ , learning rate α , replay memory size N , total iterations number T , total episodes M

- 1 Initialize replay memory $\mathcal{D} \leftarrow \emptyset$;
- 2 Initialize Θ^Q, Θ^μ at random; $\Theta^{Q'} \leftarrow \Theta^Q, \Theta^{\mu'} \leftarrow \Theta^\mu$;
- 3 Initialize $env \leftarrow EnvironmentInstance$;
- 4 **for** $m \leftarrow 1$ **to** M **do**
- 5 $\mathcal{N} \leftarrow RandomNoiseProcess()$;
- 6 $s_t \leftarrow env.InitialState()$;
- 7 **for** $t \leftarrow 1$ **to** T **do**
- 8 $a_t \leftarrow \mu(s_t | \Theta^\mu) + \mathcal{N}_t$;
- 9 $s_{t+1}, r_t \leftarrow env.step(s_t, a_t)$;
- 10 $StoreTransition(\mathcal{D}, (s_t, a_t, r_t, s_{t+1}))$;
- 11 $L \leftarrow 0$;
- 12 $P \leftarrow 0$;
- 13 **for** $j \leftarrow 1$ **to** k **do**
- 14 $\tau = (s_t, a_t, r_t, s_{t+1}) \sim UniformSample(\mathcal{D})$;
- 15 $Q_{target} \leftarrow Q'(s_{t+1}, \mu'(s_{t+1} | \Theta^{\mu'}), \Theta^{Q'})$;
- 16 $Q_{value} \leftarrow Q(s_t, a_t, \Theta^Q)$;
- 17 $\mathcal{L}(\Theta^Q) \leftarrow (r_t + \gamma \times Q_{target} - Q_{value})^2$;
- 18 $L \leftarrow L + \mathcal{L}(\Theta^Q)$;
- 19 $P \leftarrow P + \nabla_a Q(s, a | \theta^Q)|_{s=s_t, a=\mu(s_t)} \nabla_{\theta^\mu} \mu(s | \theta^\mu)|_{s=s_t}$;
- 20 $\Delta \leftarrow \frac{1}{k} L \times \nabla_{\Theta^Q}$;
- 21 $\Theta^Q \leftarrow \Theta^Q + \alpha \times \Delta$;
- 22 $\Theta^\mu \leftarrow \Theta^\mu + \alpha \times \nabla_{\Theta^\mu} J \approx \frac{1}{k} P$;
- 23 $\Theta^{Q'} \leftarrow \tau \Theta^Q + (1 - \tau) \Theta^{Q'}$;
- 24 $\Theta^{\mu'} \leftarrow \tau \Theta^\mu + (1 - \tau) \Theta^{\mu'}$;
- 25 $s_t \leftarrow s_{t+1}$;

Algorithm 4 DDPG – deep deterministic policy gradient

4.3 Improving sample data efficiency in experience replay

According to Schaul et al. (2016), an agent can learn more efficiently from some experiences than others, and some experiences may become more relevant as the agent approximates an optimal policy. Moreover, the experience replay from a uniform sampling does not consider the relevance of the experiences for the agent learning, usually repeating them at the same frequency they occurred. Given this, the authors investigated the effects of prioritization of experiences with studies in a proper environment (which presents exploration

challenges with rare rewards), resulting in the proposition of the Prioritized Experience Replay (PER) method. The authors initially investigated the effects of prioritizing experiences in reducing the number of updating steps that a Q-Learning agent needs to learn the Q-function, comparing the results using: (i) a uniform sample; (ii) an oracle that achieves the best results; and (iii) a greedy sampling strategy. This greedy strategy stores the last TD-error value along with each transition in the replay memory and replays the ones with the highest absolute value of the TD-error to update the Q-function. They verified that it reduced the number of update steps compared to the uniform sampling but presented several issues. Because it only updates the TD-error of the replayed transitions, it may not replay those transitions initially associated with low TD-error values for a long time or until they are discarded due to a replay buffer with a size constraint. Moreover, replaying experiences with high and slowly decreasing TD-errors often causes a loss of diversity. That may lead the model to overfit, besides being sensitive to noise spikes (e.g., when the rewards are stochastic). Therefore, they proposed a stochastic sampling method that combines greedy prioritization and uniform random sampling by defining the sample probability based on a transition's priority value. According to Eq. 34, the probability of sampling a transition j is

$$P(j) = \frac{p_j^\alpha}{\sum_i p_i^\alpha}, \quad (34)$$

where $p_j > 0$ is the priority of transition j and α defines how much of this value to use (i.e., $\alpha = 0$ to uniform sampling). Nevertheless, experiences' prioritizing introduces bias because it changes the probability distribution on which stochastic updates depend. Therefore, the authors proposed an approach to correct the bias called importance-sampling through a weight w_j (applied in the Q-function update) given by Eq. 35,

$$w_j = \left(\frac{1}{N} \times \frac{1}{P(j)} \right)^\beta \quad (35)$$

in which N represents the size of the replay buffer and compensates for the nonuniform probabilities $P(i)$ if $\beta = 1$. Based on the hypothesis that it is possible to ignore small values of bias since it impacts more as convergence approaches, the authors exploit the flexibility of annealing the amount of importance-sampling correction over time by (linearly from an initial value) making $\beta = 1$ only at the final of training. Finally, Schaul et al. (2016) combined prioritizing replay, stochastic sampling with priority values, and importance sampling to define the method PER (see Algorithm 5). They replaced the uniform sample in DDQN, achieving new state-of-the-art results in learning to play Atari 2600 games in ALE.

input: batch size k , learning rate η , discount factor γ , replay memory size N , exponents α and β , total iterations number T , ϵ -Greedy probability ϵ

- 1 Initialize replay memory $\mathcal{D} \leftarrow \emptyset$, $t \leftarrow 0$, priority $p_1 \leftarrow 1$;
- 2 Initialize Θ at random; Initialize $\Theta' = \Theta$;
- 3 Initialize $env \leftarrow EnvironmentInstance$; $s_t \leftarrow env.InitialState()$;
- 4 **for** $t \leftarrow 1$ **to** T **do**
- 5 $a_t \leftarrow \epsilon\text{-Greedy}(s_t, \epsilon, \Theta)$;
- 6 $s_{t+1}, r_t \leftarrow env.step(s_t, a_t)$;
- 7 $p_t \leftarrow \max_{i < t} p_i$;
- 8 $StoreTransition(\mathcal{D}, (s_t, a_t, r_t, s_{t+1}), p_t)$;
- 9 **if** $t \bmod tf == 0$ **then**
- 10 **for** $j \leftarrow 1$ **to** k **do**
- 11 $P(j) = \frac{p_j^\alpha}{\sum_i p_i^\alpha}$;
- 12 $\tau = (s_t, a_t, r_t, s_{t+1}) \sim PrioritizedSampling(\mathcal{D}, P(j))$;
- 13 $w_j \leftarrow \frac{N \cdot P(j)^{-\beta}}{\max_i w_i}$;
- 14 $Q_{target} \leftarrow Q(s_{t+1}, argmax_a Q(s_{t+1}, a, \Theta), \Theta')$;
- 15 $Q_{value} \leftarrow Q(s_t, a_t, \Theta)$;
- 16 $\mathcal{L}(\Theta) \leftarrow r_t + \gamma \times Q_{target} - Q_{value}$;
- 17 $p_j \leftarrow |\mathcal{L}(\Theta)|$;
- 18 $\Delta \leftarrow \Delta + w_j \times \mathcal{L}(\Theta) \times \nabla_{\Theta} Q_{value}$;
- 19 $\Theta \leftarrow \Theta + \eta \times \Delta$;
- 20 **if** $t \bmod utf == 0$ **then**
- 21 $\Theta' \leftarrow \Theta$;
- 22 $\Delta \leftarrow 0$;
- 23 $s_t \leftarrow s_{t+1}$;

Algorithm 5 DDQN with PER using proportional prioritization

According to Schaul et al. (2016), the use of a replay buffer presents two main challenges: (i) the selection of which experiences to store; and (ii) the picking of which ones to repeat. They addressed the second when they proposed the PER, assuming that the content of the memory was beyond their control. Novati and Koumoutsakos (2019), Zha et al. (2019), and Sun et al. (2020) also dealt with the second case seeking to make ER more optimized and data-efficient based on how to sample transitions to improve the current learning policy. In turn, Neves et al. (2022) originally approached the first case, investigating how to store the transition in a transitions memory, improving data efficiency, but mainly seeking to exploit rare and expensive experiences. For Novati and Koumoutsakos (2019), the accuracy of the updates can deteriorate when the policy diverges from past behaviors and can undermine the performance of the ER. Instead of tuning hyperparameters to slow down policy changes, they actively reinforced the similarity between current policy transitions π and past experiences μ used to compute updates, using an approach called Remember and Forget Experience Replay (ReF-ER). This skips gradients computed from experiences that are too unlikely with the current policy transitions and regulates policy changes within a trust region of the replayed behaviors. Their main objective is to control the similarity between π and μ , classifying experiences as “near-policy” or “far-policy” based on a ratio ρ of probabilities of selecting the associated action with π and that with μ . Therefore, ReF-ER limits the fraction of far-policy samples in the replay memory and computes gradient estimates only from near-policy experiences. The authors demonstrated that their approach could be applied to any off-policy method with parameterized policies (i.e., by using Deep Neural

Network – DNN) and that it allows for better stability and agent performance (compared to uniform sampling) in the main class of methods for continuous action spaces based on DPG (i.e., DDPG), Q-learning (i.e., NAF in Gu et al. (2016)), and off-policy Policy Gradients (off-PG) (Degris et al. 2012).

Zha et al. (2019) proposed an Experience Replay Optimization (ERO) framework, which aims to optimize the replay strategy by learning a replay policy (instead of applying a heuristic or rule-based strategy) whose main challenge is dealing with continuous, noised and unstable (regarding the rewards) updating of a large replay memory (usually in the tens of millions of transitions). Its objective is learning to sample experiences that could maximize the expected cumulative reward. While the agent learns a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$, ERO learns a policy $\phi : \mathcal{D} \rightarrow \mathcal{B}_i$, where $\mathcal{D}$ is the replay buffer, and $\mathcal{B}_i$ is a batch of transitions sampled from $\mathcal{D}$ at a time step i . ϕ outputs a boolean vector to guide the subset sampling, indirectly teaching the agent by defining what subset it should use to update its value functions. Then, ERO adjusts ϕ according to the return from the environment as a measure of the agent's *performance improvement*. The authors evaluated their approach by applying it to train a DDPG agent on eight continuous control tasks from the OpenAI Gym environment. They concluded their proposal is promising because it could find more “usable” experiences for off-policy agents using ER in different tasks.

Sun et al. (2020) proposed the Attentive Experience Replay (AER) to prioritize transitions (at sampling from the replay buffer) containing states more frequently observed by the current policy, based on the idea that some states in past experiences may become rarely revisited once the policy is continually updated and may not contribute to or even harm the overall performance of the current policy. The authors considered the similarity between past and current transition states as a measure of frequency and prioritization criteria. For the authors, some experiences in the replay buffer might become irrelevant to the current policy, and others may contain states that the current policy would never visit. Another supposition from Sun et al. (2020) is that some transitions from the past might contain states that would never be visited by current policy, and optimization over these states might not improve the overall performance of current policy and can undermine the performances of frequently visited states. The authors evaluated the results of applying AER in the off-policy algorithms DQN, DDPG, Soft Actor-Critic (SAC) (Haarnoja et al. 2018), and Twin Delayed Deep Deterministic Policy Gradient (TD3) (Fujimoto et al. 2018), and compared with uniform sampling and PER. They used the OpenAI Gym task ecosystem (Brockman et al. 2016).

Neves et al. (2022) proposed a method named *COMPact Experience Replay* (COMPER) to improve the model of experience memory and make ER feasible (and more efficient) using smaller amounts of data. The authors demonstrated that it is possible to produce sets of similar transitions and explore them to build a reduced transitions memory, performing successive updates of their Q-values and learning their dynamics through a Long-short Term Memory (LSTM) network. They also used this same LSTM network to approximate the target value at the TD-learning. According to the authors, this augments the odds of a rare transition being observed compared to a sampling performed on a large replay buffer and makes the updates of the value function more effective. The authors presented a complete analysis of the memories' behavior, along with detailed results for 100,000 frames and about 25,000 iterations with a small experience memory on eight challenging 2600 Atari games in the Arcade Learning Environment (ALE), demonstrating that COMPER can approximate a

good policy from a small number of frame observations using a compact memory and learning the similar transitions' sets dynamics using a recurrent neural network.

COMPER (see Algorithm 6) uses ER and TD-learning to update the Q-value function $Q(s, a)$. However, it does not just construct a replay buffer. Instead, COMPER samples transitions from a much more compact structure named *Reduced Transition Memory* ($\mathcal{RTM}$). To achieve that, COMPER first stores the transitions together with estimated Q-values into a structure named *Transition Memory* ($\mathcal{TM}$), which is similar to a traditional replay buffer, except for the presence of the Q-value and the identification and indexing of *Similar Transitions Sets* ($\mathcal{ST}$). After that, the similarities between the transitions stored in $\mathcal{TM}$ can be explored to generate a more compact version of it – the $\mathcal{RTM}$. Then, the transitions $(s_t, a_t, r_t, s_{t+1}) \sim U(\mathcal{RTM})$ are drawn uniformly from $\mathcal{RTM}$ and used to minimize the following loss function,

$$\mathcal{L}_{COMPER}(\Theta_i) = \mathbb{E}_{\tau_t=(s_t, a_t, r_t, s_{t+1}) \sim U(\mathcal{RTM})} \left[(r_t + \gamma QT(\tau_t, \Omega) - Q(s_t, a_t, \Theta_i))^2 \right], \quad (36)$$

in which $Q(s_t, a_t, \Theta_i)$ is a Q-function approximated by a CNN parameterized by Θ_i at i -th iteration. $QT(\tau_t, \Omega)$ is a Q-target function approximated by an LSMT and parameterized by Ω . This function provides the target value and is updated in a supervised way from the $\mathcal{ST}$ s stored in $\mathcal{TM}$. Thus, this LSTM is also used to build a model to generate the compact structure of $\mathcal{RTM}$ from $\mathcal{TM}$ while seeking to learn the dynamics of $\mathcal{ST}$ s to provide better target values at the next agent update step.

input: batch size k , learning rate α , training frequency tf , number of states to be observed sn , ϵ -Greedy probability ϵ , update target frequency utf , discount factor γ , similarity threshold value δ

- 1 Initialize $\mathcal{TM} \leftarrow \emptyset$, $\mathcal{RTM} \leftarrow \emptyset$, $t \leftarrow 0$, $\Delta \leftarrow 0$; Initialize Θ , Ω ;
- 2 Initialize $env \leftarrow EnvironmentInstance$, $run \leftarrow True$, $countframes \leftarrow 0$;
- 3 $s_t \leftarrow env.InitialState()$;
- 4 $a_t, Q_t \leftarrow \epsilon\text{-Greedy}(s_t, \epsilon, \Theta)$;
- 5 **while** run **do**
- 6 $s_{t+1}, r_t \leftarrow step(s_t, a_t)$;
- 7 $\mathcal{TM}.StoreTransition([\tau(s_t, a_t, r_t, s_{t+1}), Q_t], \delta)$;
- 8 $s_t \leftarrow s_{t+1}$;
- 9 $t \leftarrow t + 1$;
- 10 **if** $t \bmod tf == 0$ **then**
- 11 **if** $\mathcal{RTM} == \emptyset$ **or** $t \bmod utf == 0$ **then**
- 12 $\Omega \leftarrow Train(QT, \mathcal{TM})$;
- 13 $\mathcal{RTM} \leftarrow ProduceRTM(QT, \mathcal{TM}, \Omega)$;
- 14 **for** $i \leftarrow 1$ **to** k **do**
- 15 Sample uniformly $\tau' = (s_t, a_t, r_t, s_{t+1})$ from $\mathcal{RTM}$;
- 16 $Q_{target} \leftarrow Forward(\tau', QT, \Omega)$;
- 17 $Q_{value} \leftarrow Forward(s_t, Q, \Theta)[a_t]$;
- 18 $\mathcal{L}(\Theta) \leftarrow r_t + \gamma \times Q_{target} - Q_{value}$;
- 19 $\Delta \leftarrow \Delta + \mathcal{L}(\Theta) \times \nabla_{\Theta} Q_{value}$;
- 20 $\Theta \leftarrow \Theta + \alpha \times \Delta$;
- 21 $\Delta \leftarrow 0$;
- 22 $countframes \leftarrow countframes + env.framesnumber$;
- 23 **if** $env.ReachedFinalState()$ **then**
- 24 $run \leftarrow (countframes \leq sn)$;
- 25 **if** run **then**
- 26 $env.ResetStates()$;
- 27 $s_t \leftarrow env.InitialState()$;
- 28 $a_t, Q_t \leftarrow \epsilon\text{-Greedy}(s_t, \epsilon, \Theta)$;

Algorithm 6 COMPER – *COMPact experience replay*

At each training time-step t , the authors define a transition by a tuple $\tau_t = (s_t, a_t, r_t, s_{t+1})$. Two transitions $\tau_{t_1} = (s_{t_1}, a_{t_1}, r_{t_1}, s_{t_1+1})$ and $\tau_{t_2} = (s_{t_2}, a_{t_2}, r_{t_2}, s_{t_2+1})$, $t_1 \neq t_2$, are similar ($\tau_{t_1} \approx \tau_{t_2}$) when the distance (e.g., Euclidean distance) between τ_{t_1} and τ_{t_2} is less than a threshold, that is, $\mathcal{D}(\tau_{t_1}, \tau_{t_2}) \leq \delta$, in which δ is a threshold value of distance (or similarity). The N transitions that occurred up to a time instant are stored in $\mathcal{TM}$ and can be identified as subsets of similar transitions $\mathcal{ST}$ when the similarity condition is satisfied. In addition, they are stored throughout subsequent agent training episodes and are identified by a unique index. Therefore, the authors define $\mathcal{TM} = \{[T^i, \mathcal{ST}_i] \mid i = 1, 2, 3, \dots, N_{ST}\}$, in which N_{ST} is the total number of distinct subsets of similar transitions, T^i is a unique numbered index and $\mathcal{ST}_i$ represents a set of similar transitions and their Q-values. Thus,

$$\mathcal{ST}_i = \{[\tau_{i(1)}, Q_{i(k)}] \mid 1 \leq k \leq N_{ST}^i\} \quad (37)$$

in which N_{ST}^i represents the total number of similar transitions in the set $\mathcal{ST}_i$. Thus, $\tau_{i(1)}$ corresponds to some transition τ_{t_j} , $j \in \{1, \dots, N_{ST}^i\}$ and is the representing transition of similar transitions set $\mathcal{ST}_i$ (e.g., the first one), and $Q_{i(k)}$ is the Q-value corresponding to some transition τ_{t_j} , $j \in \{1, \dots, N_{ST}^i\}$ such that $\tau_{i(1)} \in \mathcal{ST}_i$ and $\tau_{i(1)} \approx \tau_{i(k)}$, $1 \leq k \leq N_{ST}^i$. Therefore, $\mathcal{TM}$ can seem as a set of $\mathcal{ST}$ s. A single representative transition for each $\mathcal{ST}$ can be generated together with the prediction of their next Q-value from an explicit model of $\mathcal{ST}$ using the LSTM. This way, from $\mathcal{TM}$, one can produce a $\mathcal{RTM}$ in which τ_i' is the transition that represents all the similar transitions so far identified in $\mathcal{ST}_i$, so that $\mathcal{RTM} = \{[\tau_i'] \mid i = 1, 2, 3, \dots, N_{ST}\}$. Unlike $\mathcal{TM}$, $\mathcal{RTM}$ does not take care of sets of similar transitions since each τ_i' is unique and represents all the transitions in a given $\mathcal{ST}_i$. According to the authors, it gives the transitions stored in $\mathcal{RTM}$ the chance of having their Q-values re-estimated. Besides, sampling from $\mathcal{RTM}$ increases the chances of selecting rare and very informative transitions more frequently, at the same time, that helps increase diversity (because of variability in each sample).

One could observe in Algorithm 7 that COMPER slightly modifies the ϵ -Greedy algorithm to return the estimation of the Q-value together with the action that maximizes it.

input : State s , ϵ -Greedy probability ϵ , CNN parameters Θ
output: action a , Q-value Q

```

1 qValues  $\leftarrow$  CNN.Forward( $s, \Theta$ );
2 if randomNumber  $>$   $\epsilon$  then
3   |  $a \leftarrow$  argmax(qValues);
4   |  $Q \leftarrow$  qValues[ $a$ ];
5 else
6   |  $a \leftarrow$  random(#actions);
7   |  $Q \leftarrow$  qValues[ $a$ ];
8 end
9 return  $a, Q$ ;
```

Algorithm 7 COMPER ϵ -Greedy

One of the main contributions of Experience Replay (ER) is to reduce nonstationarity and decorrelate the agent's updates, contributing to the stabilization when using deep neural networks to approximate the value functions. However, how it stores and samples the agent's experiences using the experience replay memory limits its use to off-policy reinforcement learning algorithms. In place of ER, Mnih et al. (2016) proposed using asynchro-

nous gradient descent to optimize deep neural networks and train several agents in parallel in multiple instances of the environment. According to the authors, this parallelism also decorrelates the agents' data because, at each time step, the parallel agents will probably be experiencing various and different states and can explicitly use different exploration policies to maximize their diversity. Moreover, by running different exploration policies in multiple threads, the overall updating changes by multiple actor-learners applying online updates in parallel are likely to be less correlated in time than a single online agent, fulfilling the role of stabilizing undertaken by ER. The authors demonstrated that their approach could be used in off-policy and on-policy algorithms by presenting multithreaded asynchronous variants of Q-learning, Sarsa, and Advantage Actor-Critic methods. Their best-evaluated algorithm called the Asynchronous Advantage Actor-Critic (A3C) surpassed the state-of-the-art (at its publication time) on the Atari 2600 domain in ALE and reduced the training time because that is roughly linear in the number of parallel actor-learners. The authors also evaluated A3C on the MuJoCo physics simulator domain (Todorov et al. 2012).

4.4 Combining benefits in ensemble methods

Many relevant improvements in DQN-based methods approach different aspects. Although DDQN addresses the overestimation bias of Q-learning and (as a consequence) of DQN, PER improves data efficiency in experience replaying. The Dueling Network improved the generalization across actions by representing state values and action advantages separately. A3C shifts the bias-variance trade-off by learning from multistep bootstrap targets and helps to propagate newly observed rewards faster to earlier visited states. Distributional Q-learning learns a categorical distribution of discounted returns instead of estimating the mean. Noisy DQN uses stochastic network layers for exploration. Given this, Hessel et al. (2018) investigated how to combine these different but complementary ideas, using ER, into an ensemble approach called Rainbow, which achieved state-of-the-art on 57 Atari 2600 games in ALE (Bellemare et al. 2013) concerning data efficiency and final performance.

The authors adapted the PER strategies to use the KL loss of the Distributional Q-Learning, replaced the one-step distributional loss with a multistep variant, and defined the target distribution by contracting the value distribution in S_{t+n} and shifting it by the truncated n -step discounted return. They combined multistep distributional loss with Double Q-Learning and used the greedy strategy to select and evaluate the action in S_{t+n} using the target and online networks. They also adapted the dueling network architecture for use with return distributions so that the output of a shared state representation layer is fed into a value stream and an advanced stream designed to output distributional values that are combined as in Dueling Networks and then passed through a softmax layer to obtain the normalized parametric distributions used to estimate the returns' distributions. Finally, they replaced all linear layers with equivalent noisy layers and used factorized Gaussian noise (Fortunato et al. 2018) to reduce the number of independent noise variables. An open-source variation of Rainbow is available in the framework for RL agents development called Dopamine (Castro et al. 2018), which differs from the original Rainbow (Hessel et al. 2018) by not including DDQN, dueling heads or noisy networks. It uses the n -step returns, which is identified by Fedus et al. (2020) as a critical element to improve the agent performance when using a larger replay buffer (i.e., 10 million experiences instead of the classical 1 million limited size). The n -step returns updates the Q-value function from an n -step target value rather

than one-step so that the target side of Q-learning would be changed from Eqs. 38, 39. The authors interpret it as an interpolation between estimating targets in Monte Carlo (MC) methods as $\sum_{k=0}^T \gamma^k r_{t+k}$ (a discussion can be found in Sutton and Barto (2018)) and single-step TD-learning, balancing the low bias but high variance of the MC targets and the low variance but high bias of TD(0) (see Sect. 2).

$$r + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \quad (38)$$

$$\sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_a Q(s_{t+n}, a) \quad (39)$$

Kaiser et al. (2019) presented a model-based deep reinforcement learning algorithm with a video prediction model named SimPLe, which performed well after just 102,400 interactions (that correspond to 409,600 frames on ALE and about 800,000 samples from the video prediction model) and compared their results with the ones obtained by Rainbow (Hessel et al. 2018). They aimed to show that planning with a parametric model allows for data-efficient learning on several Atari video games. In that sense, van Hasselt et al. (2019) proposed a broad discussion about model-based algorithms and experience replay, pointing out its commonalities and differences, when to expect benefits from either approach, and how to interpret prior works in this context. They set up experiments in a way comparable to Kaiser et al. (2019). They demonstrated that in a like-for-like comparison, Rainbow outperformed the scores of the model-based agent with less experience and computation. Rainbow used a total number of 3.2 million replayed samples, and SimPLe used 15.2 million. Łukasz Kaiser et al. (2020) presented their final published paper comparing SimPLe and Rainbow on the number of iterations needed to achieve the best results. SimPLe achieved the best game scores on half of the game set. However, the authors state that one of the SimPLe limitations is that its final scores are, on the whole, lower than the best state-of-the-art model-free methods.

5 Challenges and trends in experience replay

The challenges we found in the literature, from the early propositions until the most recent research, allowed us to identify a set we could consider essential problems, such as the ones Experience Replay proposed to solve. However, there are different classes of relatively recent issues arising from previously proposed approaches to open problems, such as those that PER proposed to address at the expense of possible bias or the many methods that suffer from catastrophic forgetting by benefiting from the ER, for example. This section identifies some relevant general problems in the Experience Replay domain (despite the many benefits of each approach in the literature), as presented in Table 1, and selects some to bring exciting and in-depth discussions of the literature.

Table 1 Main challenges in reinforcement learning with experience replay

Problem	Description
Replay buffer size	It is related to complex challenges, like: (i) FIFO-like buffers can lead to a loss of experiences because of limited size; (ii) Uniform sampling in FIFO-like buffers may overwrite many relevant experiences because it does not differentiate experiences; (iii) Small buffers produce fresher but biased samples; (iv) Large buffers produce more diverse but outdated samples. We consider research works aiming to overcome the limitations that arise from the buffer size
Exploration efficiency	Imposes a difficult problem (especially in continuous action spaces) in regulating trade-offs between expanding knowledge by exploring actions and refining knowledge by exploiting it. It is closely related to bias and diversity and directly impacts the agent's performance. We consider works that concentrate on improving or proposing strategies for better exploration and exploitation
Sampling efficiency	Some experiences contribute more to the agent's learning than others, and some experiences may become more relevant as the agent approximates an optimal policy. The same batch of samples may be more important than others. Not exploring this in some way may negatively impact relevant aspects of the agent's learning. We consider works that focus on making relevant sampling
Data efficiency	Using the agents' experiences in a data-efficient way implies thinking about extracting information directly from their data (such as using RNN), modeling transitions and memories (such as using many buffers, compacting memories, or proposing new data structures), and organizing and storing the experiences. We consider works that directly exploit the transition's data (i.e., its data values) or the memory structure
Catastrophic forgetting	Leads to a loss of information and instability in agent training when new experiences overwrite old experiences in the multitasking or multi-agent learning scenario. It is closely related to the buffer size and data efficiency. We consider works that mainly overcome the limitations that arise from the buffer limitation in a shared scenario
Sparse rewards	This is more of a fundamental and complex problem in TD-Learning-based RL than a closely ER-related problem. It imposes on the agent to learn long-term policies until rewarded or to explore an arbitrarily large space of experiences because the immediate rewards are fundamental to guiding the optimal policy approximation. However, there are notable results in approaching this problem by employing strategies of ER

5.1 Replay buffer size

Relevant research works have sought to understand the effects of replay buffer size (either small or large) on the performance of reinforcement learning agents that use ER. For example, since Mnih et al. (2015), all DQN-based methods use a fixed replay memory size of one million transitions, and variations in the buffer's size are still understudied in this class of methods (Liu and Zou 2018). Zhang and Sutton (2017) presented an empirical study on ER, demonstrating that a large replay buffer can harm agent performance and that its size is a very important hyperparameter neglected in the literature. They proposed a method to minimize the negative influence of a large replay buffer called Combined Experience Replay (CER) consisting of adding the last transition to the sampled batch before using it in agent training. They hypothesized a trade-off between the data quality and the data correlation, as smaller replay buffers make data fresher but highly temporal correlated. At the same time, neural networks often need independent and identically distributed (i.i.d.) data. Data sampling from larger replay buffers tends to be uncorrelated but outdated. A full replay buffer adopting a FIFO strategy (i.e., as a queue) will impact agent learning. However, according to Neves et al. (2022), if we assume that different transitions from many stochastic episodes carried out over a nondeterministic environment will be stored and sampled many times from a smaller replay buffer (but not explicitly size-limited), this buffer tends to be less correlated along with the time while the buffer size increases.

According to Fedus et al. (2020), it is necessary to investigate the behavior of ER concerning the interrelated effect of variations in its hyperparameters, since their effects may not only be individual but come from their joint variation. The authors studied the relationship between the size of the replay buffer, the *replay capacity*, and the time that a transition (which represents a policy at some moment in the learning process) remains in memory, which they call the *age of a policy*. Thus, the replay capacity is associated with state-action coverage, while the age of a policy (represented by its respective transition in memory) is related to its distance from the current policy learned (represented by the most recent transitions). It is possible to explore this relationship through an element called *replay ratio*, which refers to the number of agent interaction steps with the environment for each step of updating the value function. For example, DQN (Mnih et al. 2015) performs an update step (from the memory of the transitions) for every 4 iteration steps, which means a replay ratio of 0.25. Thus, their primary objective was to understand how the agent behavior changes as the replay ratio varies. Objectively, the authors defined the age of the policy by the number of value function update steps performed since the storage of the transition (which represents the policy) in the buffer and the replay capacity by the total number of transitions stored. So, with a replay capacity of 1 million transitions and a replay ratio of 0.25, the oldest policy age is 250,000 function update steps. In this relationship, increasing the buffer size also increases the replay capacity and the (possible) age of the policy. In this way, the replay ratio remains constant. However, fixing the potential age of the oldest policy requires storing more transitions by the current policy to increase the replay capacity. In other words, it is necessary to decrease the number of function updates per number of interactions with the environment, which increases the number of agent interaction steps to store a more significant number of transitions from the current policy, which results in a decrease in replay ratio. On the other hand, keeping the replay capacity fixed (fixing the buffer size) decreases

the age of the oldest policy but also requires more transitions, which will reduce the replay ratio.

Still in Fedus et al. (2020), the authors performed experiments using 14 Atari 2600 games in ALE using the Dopamine version of Rainbow (Castro et al. 2018), demonstrating that Rainbow performance consistently improves when replay capacity increases which is generally related to the age reduction of the oldest policy. When one fixes the policy's age, the performance increases with the growth of replay capacity, and this trend remains regardless of the value defined for the policy's age. However, the magnitude of the increase in performance depends on this value. For the authors, this may occur because a larger state-action coverage capacity can reduce the chances of overfitting over a smaller subset of transitions. On the other hand, when the replay capacity is fixed, the performance tends to improve with the reduction of the age of the oldest policy. Based on the idea that the age of an old policy distances it from the current policy and that the age depends on the replay ratio and replay capacity, the authors claimed that the results of their experiments suggest that learning from policies (sampling transitions) closer to the current policy can increase performance because the agent thus explores transitions with a more significant potential of return (Sun et al. (2020) also explored this hypothesis). An exception observed by the authors occurred when they fixed the replay capacity at 10 million and reduced the age of the oldest policy from 2.5 million to 250 thousand, which, according to them, can be explained by the decrease in the agent score in two games: Montezuma Revenge and PrivateEye, which consist of two environments with very sparse rewards and challenging to explore. In these environments, because of the sparse rewards that demand learning long-term policies, the agent could not accumulate rewards when the authors reduced the potential age of the policies and concentrated the transition samples in those closer to the current policy. The authors also noted that increasing the buffer size while maintaining a fixed replay ratio leads to a performance improvement that may vary due to the interaction between the gain in replay capacity and the loss from accumulating older policies. Thus, as the age of the policy increases, the benefit of increasing the replay capacity generally decreases. One could see that the issue with Montezuma Revenge and PrivateEye games is related to the need to learn longer-term policies, which Neves et al. (2022) also sought to address with the use of recurrence on sets of similar transitions and sampling from a smaller memory whose size is not explicitly limited.

When conducting experiments with DQN, the authors observed that its performance did not increase with the growth of the replay capacity, either by fixing the replay ratio or by fixing the age of the policy, contrary to what they observed with the Rainbow version of Dopamine. This version is (basically) the DQN with the addition of PER, C51, and n-step returns, plus the replacement of the RMSProp optimization method with ADAM (Kingma and Ba 2015). Compared to the original Rainbow, that one does not include DDQN, dueling, or noisy networks. Therefore, the authors created four DQN variants in which each received the addition of only one of those components to investigate which interacted with the increase in replay capacity (from 1 million to 10 million transitions) to generate the performance gain observed in Rainbow. The results showed that the only independently added component that led to a considerable performance improvement with an increase in the replay capacity in the DQN was the n-step returns. This increase also occurs when the policy age is fixed instead of the replay ratio. By removing the n-step return from Rainbow, they verified that the agent did not benefit from the increase in replay capacity and that removing other components does not prevent performance gains. It suggests that n-step

returns are the only critical component for performance gain with increased replay capacity. They also observed that the use of n -step returns when there are few transitions in the buffer worsens the performance of the DQN, suggesting that the performance gain due to its use only occurs when using larger transition buffers. They also noticed that only adding PER to the DQN does not significantly increase performance when the replay capacity is considerably large. One can see in Sect. 4 how it distributes the weighting obtained as a function of the magnitude of the temporal difference errors in the transition memory and how its possible loss of capacity to contribute to the increase in performance may be related to the buffer size. The authors also carried out experiments with two variations of the DQN using the n -step returns and offline trained from a buffer of 200 million transitions (corresponding to the total frames used to evaluate agents in the literature) to verify if the performance gain persists while having bigger and bigger buffers, keeping the replay ratio fixed and making the buffer have older policies. Those transitions come from another agent and do not have their Q-value estimates updated during the training of the current DQN agent. Still, the authors observed a consistent increase in performance.

Fedus et al. (2020) concluded that increasing the replay capacity and reducing the age of the oldest policy increases agent performance and that the n -step returns process is the only element used in Rainbow capable of taking advantage of expanding the replay capacity. Investigating the relationship between n -step returns and Experience Replay, the authors observed that the replay capacity can mitigate the variance of n -step returns, which partially explains the performance increase. The authors highlighted essential aspects regarding the interaction between the learning algorithms and the mechanisms that generate the training data (i.e., the memory of transitions): the distance between the oldest policy and the current policy the agent is learning (which is a classic problem in RL with ER), the state-action coverage space, the correlation between transitions (also explored in Neves et al. (2022) and Sun et al. (2020), but through another form of exploitation of their similarities), and the cardinality of the distribution support. For the authors, these aspects can be challenging to control separately and independently because typical algorithmic adjustments can affect several of them simultaneously. In future works, the authors pointed to studying how to untangle those different aspects to obtain agents capable of efficiently scaling in performance with the increase of available data, thus investigating how these aspects of Experience Replay interact with other classes of off-policy and multi-step reinforcement methods.

5.2 Exploration efficiency

Fortunato et al. (2018) approached the relevant problem of exploration, proposing a method called NoisyNet to learn neural networks' weight perturbations and using this to drive exploration. The authors based this on the idea that a single change in the weight vector can induce consistent and very complex state-dependent changes in the policy over multiple time steps instead of adding decor-related and state-independent noise, such as in ϵ -greedy. According to the authors, most exploration heuristics, such as ϵ -greedy and entropy regularization, may not produce large-scale behavioral patterns necessary for efficient exploration in many environments. The methods of *optimism in the face of the uncertain* are often limited to small states-action spaces or linear function approximations. The intrinsic motivation methods augment the environment's reward signal with an additional term to reward novel discoveries, and many research works have proposed different forms of such terms. These

methods separate the generalization processes from the exploration, using elements like metrics for intrinsic rewards and importance values, weighted relative to the environment reward, directly defined by the researcher and not learned for the agent's interaction with the environment. Some evolutionary or black-box methods explore the policy space but require many prolonged interactions and usually are not data-efficient, requiring a simulator to allow many policy evaluations. The NoiseNet is a neural network that uses the gradient from the agent loss function (by gradient descent) to learn a parameter (alongside the other parameters of the agent) that defines the variance of the agent's neural network weights perturbations sampled from a noise distribution (which the authors called the energy of the injected noise). Therefore, the algorithm injects noise into the parameters and tunes its intensity automatically, defining what the author called the energy of the injected noise. The authors evaluated NoiseNet versions of DQN, Dueling Networks, and A3C (Mnih et al. 2016) on 57 Atari 2600 games in the Arcade Learning Environment (Machado et al. 2018), and their results demonstrated that their agents achieved superhuman performance.

5.3 Sampling efficiency

While Fedus et al. (2020) demonstrated that the use of n -step return is a critical element for Rainbow's performance, ablative studies in Fujimoto et al. (2020) demonstrated that in DQN, this element is PER. According to Fujimoto et al. (2020), although widely used, this prioritization method lacks a critical theoretical foundation, which they formally developed in their work. The expected loss over the distribution of a sample of transitions determines the gradient used for neural network optimization; therefore, when PER causes a bias in this distribution, it effectively modifies this gradient and influences the optimization process. The authors demonstrated that the gradient expected from the minimization of a loss function over a nonuniform distribution is equal to the gradient obtained from another distinct but equivalent loss function minimized over a uniform distribution and that one can use this relationship to transform any loss function into a prioritized sampling scheme with a new function and vice versa. In this way, that transformation allows a concrete understanding of the benefits of nonuniform samplings, such as in the PER, and provides a tool for designing new prioritization schemes. In this sense, the authors pointed out three relevant aspects. The first one is that the loss function and the prioritization strategy must be linked, and the design of prioritized sampling methods should not be considered independently of the loss function since this allows verifying the correctness of these methods by transforming the loss in its equivalent on a uniform sampling and verifying if it produces the same results. According to the authors, if using a proper loss function, even PER may not be biased, even if not using its importance sampling calculation (presented in Sect. 4). The second aspect is the reduction of variance, which allows a deeper understanding of the prioritization benefits since it is related to the expected gradients. This variance can be reduced by a loss function over uniform sampling and carefully choosing a prioritization scheme defined in conjunction with a corresponding loss function. A third interesting aspect is that the formulations demonstrated by the authors suggest that some of the benefits obtained by prioritized sampling come from the changes generated in the expected gradient and not from the prioritization itself.

The authors focused their ablative analysis around three loss functions and how they relate to uniform and nonuniform sampling, using a prioritization scheme and comparing

their expected gradients. They demonstrated how to carry out the proposed transformations and reduce the gradient variance by applying gradient steps of the same size instead of interspersing larger and smaller steps, pointing out a simple way to minimize the variance of any loss function while keeping the expected gradient unchanged. In this way, the authors took PER as a basis and derived an equivalent loss function for uniform sampling, allowing them to point out corrections and possible improvements in the method. Among other things, they demonstrated that when PER uses Mean Squared Error (MSE), including some subsets with Huber Loss, it optimizes the loss function over TD-error to a power higher two, indicating that it can favor outliers in its estimation of expected target values in the temporal difference rather than learning the mean. According to the authors, the bias in PER may cause its low performance in continuous learning algorithms that depend on the MSE. In addition, they demonstrated that the importance sampling ratios used by PER can be absorbed in the loss function itself, simplifying the algorithm. PER uses importance sampling in weighting the loss function to reduce the bias introduced by prioritization. However, it is no longer biased when using the MSE and setting the hyperparameter $\beta = 1$ (see Sect. 4). As the expected gradient can absorb the prioritization, PER can be unbiased even without using importance sampling if the expected gradient is still meaningful. Based on their findings, they proposed a new prioritization scheme called Loss-adjusted Prioritized (LAP) Experience Replay, which simplifies PER by removing the unnecessary importance sampling ratios and setting the minimum priority to one, which reduces bias and the likelihood of dead transitions with near-zero sampling probability. They also proposed an equivalent loss function for uniform sampling called Prioritized Approximation Loss (PAL). It resembles a weighted variant of the Huber Loss function and produces the same expected gradient as LAP. The authors showed that when the variance in the prioritization is minimal, PAL can be used instead of LAP in a simple and computationally efficient way to train neural networks that estimate Q-values and that the loss function and the form of prioritization are closely linked. They pointed out that the loss defined by PAL is never a power greater than two, meaning there is no more outlier bias from PER.

The authors used the Atari 2600 games and the MuJoCo continuous control tasks through the OpenAI Gym environment to empirically verify the effects produced by the LAP and PAL methods. In the first case, they combined their methods with the DDQN and compared them with the original DDQN and the DDQN combined with PER. In the second case, they combined their methods with the TD3 algorithm and compared it with the original TD3, with TD3 combined with PER and SAC algorithms. According to the authors, there was no consistent difference between using LAP or PAL in TD3, which means that prioritization has little benefit in this domain and that the expressive performance gain comes from the change in the expected gradient. Consequently, they showed that it is possible to replace nonuniform sampling by modifying only the loss function. PER adds little benefit to TD3, which is consistent with the authors' ablative analysis, which showed that using MSE with PER introduces bias. LAP also produced excellent results in the Atari games, surpassing the performance added by PER in 9 of 10 games, while using PAL led to a worse performance in 6 games. According to the authors, this suggested that prioritization plays a more significant role in this domain, considering that games depend on longer observation horizons (with longer-term policy learning) and sometimes have sparse rewards which does not mean that some improvements do not come from changes in the expected gradient. The authors considered the performance of PAL in MuJoCo tasks particularly interesting because of

the simplicity of the method. They believed that its benefits over the MSE and the original Huber Loss came from its robustness and ability to approximate better the average.

For Fujimoto et al. (2020), more research is necessary to understand better nonuniform sampling and propose new prioritization schemes. They also reinforced the sensitivity of deep learning algorithms to minor changes since they achieved considerable performance gains in well-known algorithms just by changing the loss function. For them, this suggests that works in the literature that use intense optimization of hyperparameters or algorithmic changes may be showing gains over the original algorithms due to unclear consequences and beyond the proposals of the articles.

Different ways exist to use the agent's experiences to update its value functions, either on-policy in actor-critic methods, off-policy in methods based on Q-learning, or even in evaluating target values in TD-learning. According to Sinha et al. (2022), the importance weights methods for prioritization can improve the evaluation of the target value for more prolonged traces using $TD(\gamma)$ and be used to reduce bias on values computed from off-policy experiences. In this sense, the authors proposed a method to weight experiences based on their likelihood under the stationary distribution of the current policy and justify this with a contraction argument over the Bellman evaluation operator. Their proposal aimed to encourage on-policy sampling behaviors similar to the ReF-ER but without the need to know the policy distribution. For the authors, the Distribution Correction (DisCor) method (Kumar et al. 2020) suggested not using on-policy experiences in this context, which contrasts with their proposal. However, DisCor bases its analysis on the Bellman optimality operator instead of the Bellman evaluation operator. The difference is that the first operator seeks to find the optimal Q-value, while the second aims to find the Q-value function of the current policy. The authors' objective was to improve the performance of TD learning in the approximation of functions and not to use the weights to estimate an advantage function or to reduce the bias in estimating rewards. Indeed, Sinha et al. (2022) mixed on-policy and off-policy experiences and sought to balance their variance and bias by estimating likelihood-free density ratios and using the learned ratios as weights for prioritization, respectively.

5.4 Data efficiency

On-policy methods are sometimes more effective in specific domains, such as continuous learning. However, using off-policy data produces more efficient sampling which is critical for exploring environments with rare and expensive experiences. In this sense, replaying experiences based on prioritization schemes increases sampling efficiency. However, its use in different methods and domains depends on the strategy and objective of this prioritization. For example, PER is not very effective in methods based on actor-critic because it bases its prioritization scheme on the magnitude of TD-error from the off-policy experiences stored in the buffer. In contrast, actor-critic methods seek to approximate a Q-value function induced by the current policy, in which it may be better to perform prioritized sampling to reflect on-policy experiences. Therefore, prioritization schemes can lead to considerable improvements in sampling in actor-critic methods. According to Sinha et al. (2022), it is possible to estimate the value function of a policy by minimizing the expected squared difference between the estimate of the critical function, and the estimate of its target function (actor-critic methods contain the actor function, the critic function, and each of these

has its respective target function) over a replay buffer that properly reflects the discrepancy between the two. In this way, one can consider this discrepancy as a priority when it preserves the contraction properties of the Bellman evaluation operator while being measured by the expected quadratic distance under some state-action distribution. In this sense, the authors presented proof that the stationary distribution of the current policy is the only one in which the Bellman operator is a contraction, which they then define as being a property, and proposed the use of the non-stationary distribution as the underlying distribution of the buffer of repetition, leading to the elaboration of their method of experience replay with likelihood-free weights.

Generally, there is less experience from the current policy, and because of this, its use produces estimates with high variance. On the other hand, having more experiences under other policies in the same environment leads to the introduction of bias. Therefore, the method proposed by the authors seeks to obtain their estimates of density rates from a classifier trained to distinguish different types of experience. For this, they used a smaller buffer, which contains experiences closer to the on-policy experiences, and another larger buffer to store the off-policy experiences, estimating the density rates from these buffers. These rates are then used as importance weights to update the Q-value function, encouraging more updates from more desired state-action pairs under the current policy stationary distribution, which are more present in the smaller buffer. The authors combined their approach with the methods Soft-Actor Critic (SAC) (Haarnoja et al. 2018), Data Regularized Q (DrQ) (Yarats et al. 2021) and DDQN, and compared with the use of uniform sampling, PER, and Emphasizing Recent Experience (ERE) (Wang and Ross 2019). To evaluate their approach, they used the ALE environments, the DeepMind Control Suite (DCS) (Tassa et al. 2018), and the OpenAI Gym tasks, demonstrating considerable improvements due to the introduction of their method.

Schmitt et al. (2020) proposed an importance sampling scheme for training actor-critic off-policy agents from a large replay buffer containing at least ten times more experiences than the limit of one million commonly used in the literature. As part of their work, they proposed solutions to improve stability and make the off-policy learning of those agents more efficient, even when they are learning from the experiences of other agents through a shared experience replay process. Their approach obtained state-of-the-art results in training a single agent for 200 million interaction steps on the ALE and DMLab-30 environments and in training concurrently several agents sharing the same repetition buffer. Their algorithm has two main characteristics: mixing transitions from the replay buffer with on-policy transitions and computing what the authors define as a trust region scheme.

Still according to the authors in Schmitt et al. (2020), combining Experience Replay with actor-critic algorithms is difficult due to their on-policy nature. Despite that, they proposed trust region schemes for mixing replay buffer experiences with on-policy transition data. That allowed the importance sampling method called V-trace to scale to data distributions over which its original formulation would become unstable. V-trace is a technique widely used in training actor-critic agents, which controls the variance commonly observed in naive importance sampling, but at the cost of introducing bias in the estimates. This bias is because its estimate of the v_π value function does not correspond to the expected return value for the policy π but rather for a policy $\tilde{\pi}$, which is only implicitly related to π and computed biased. Therefore, it can distance too much from π . In this way, the policy gradient is also biased so that, given a value function v^* , the V-trace does not guarantee conver-

gence to a policy π^* in off-line training, as the authors demonstrated. Given this, blending on-policy transitions can mitigate the distortion caused by V-trace bias, regularizing the Q-value estimates. A trust region scheme for the off-policy V-trace limits the sampling of off-policy transitions by rejecting highly biased transitions, aiming to provide the agent with an experience replay scheme enriched with experiences with low variance (because of V-trace) and low bias. For this, the authors defined the behavior-relevant function to classify relevant behaviors and a trust region estimator, which computes expected values from relevant experiences. This approach applied the Kullback–Leibler divergence between the target policy π and the implicit policy $\tilde{\pi}$ and is used for the policy and value estimates.

In addition to performance improvement compared to state-of-the-art algorithms, the authors showed that uniform sampling obtains results comparable to those obtained with PER. They also demonstrated that learning using only off-policy experiences without inserting recent experiences degrades performance, which also happens when using shared experiences without defining trust regions. According to the authors, their ablative experiments exhibited little benefit in using PER in actor-critic methods on the DMLab-30 environments. They highlighted that PER computes priorities on the magnitude of the TD-error which is poorly defined when sharing multi-agent experiences.

According to Kapturowski et al. (2019), increasingly complex partially observable domains have demanded considerable advances in the representation of transitions' memories and solutions based on the principles of recurrent neural networks such as LSTM, and its use has increased to overcome the challenges of these environments. Given this, the authors investigated agent training using a recurrent neural network with Experience Replay. They demonstrated its effects on parameter delay, resulting in representational deviation and recurrent state outdatedness, potentially exacerbated in distributed training environments, which leads to a loss of stability and performance during agent training. From a series of empirical studies on mitigating these issues, the authors presented their proposal called Recurrent Replay Distributed DQM (R2D2), whose significant algorithmic advances led to state-of-the-art results in Atari 2600 games in ALE and equivalent results or higher than those of state-of-the-art in DMLab-30 environment. According to the authors, R2D2 was the first to achieve these results using the same network architecture and the same hyperparameter values.

The authors modeled the environment as a Partially Observable Markov Decision Process (POMDP), defined by a tuple (S, A, T, R, Ω, O) , in which T corresponds to the transition function, Ω is the set of observations potentially received by the agent, and O maps the states to probability distributions over the observations. Thus, the agent performs an observation $o \in \Omega$ containing only partial information about the underlying state $s \in S$. An action in the environment results in a transition to a state $s' \sim T(\cdot|s, a)$, which results in an observation $o \sim \Omega(\cdot|s, a)$ and a reward $r \sim R(s, a)$. The authors then used an optimized RNN with a technique called Backpropagation Over Time (BPTT) (Werbos 1990) to learn a representation that disambiguates the real state of the POMDP. In turn, an R2D2 agent is a DQN agent with n-step returns that uses Prioritized Distributional Experience Replay and whose experiences are generated by 256 agents in parallel and used by a single learning agent. The actors use the Dueling Network (Wang et al. 2016) architecture (with an additional LSTM layer after the convolutional layers) to approximate the Q-function. Instead of storing the transitions represented by the tuples (s, a, r, s') , the algorithm stores sequences of tuples in the format (s, a, r) , with a fixed length ($m = 80$), and underlying sequences

that overlap periodically at each predefined time interval, without exceeding the limits of each episode. The authors used invertible value-function rescaling for the reward values to generate the n -step target values for the Q-value function. They also used a more aggressive prioritization scheme that employs a combination of max and mean absolute n -step TD-errors, as the mean over long sequences tends to hide larger errors, compressing the range of priorities and limiting the prioritization ability of valuable experiences.

According to the authors, to deal with partially observable environments, agents need representations of states that encode information about their trajectory of state-action pairs and their observation of the current state. According to them, the most common way to do this is to use an LSTM trained on complete trajectories stored in the replay buffer so that it can learn relevant long-term dependencies. For this, it is possible to use the zero start state to initialize the network at the beginning of the sampling sequence, which allows independent decorrelated sampling of relatively short sequences, which is essential for robust optimization. However, this forces the recurrent network to learn to retrieve useful predictions from an atypical initial recurrent state, which could limit its ability to rely on its recurrent states to learn to exploit long temporal correlations. Another possible way is to replay the entire episode trajectories, which avoids finding an inadequate initial state. However, the possibility of sequence size variations, which also depend on each environment, the high variance in network updates, and the use of highly correlated data can bring about a series of stability problems. Therefore, the authors proposed and evaluated two training strategies to measure and mitigate the harmful effects of representational deviation and the outdating of recurrent states. After comparing trained agents using each strategy in various DeepMind Lab environments, the authors identified that their combination consistently produced the smallest discrepancy in the last states of the sequence together with more robust performance improvements than when used separately.

Finally, the authors evaluated R2D2 in the 57 games of ALE and the different tasks of the DMLab-30 environment. They compared their results with those obtained by Ape-X (Horgan et al. 2018) and IMPALA (Espeholt et al. 2018). Unlike R2D2, the hyperparameters were adjusted separately for each environment. The authors pointed out that one of the most significant contributions of DQN was its ability to generalize over different environments using the same network architecture and hyperparameter values. According to them, until the date of their publication, no other work had maintained this kind of generalization pattern, using the same architecture and hyperparameters in both the ALE and DMLab-30 environments. The authors stated that Rainbow and IQN (Dabney et al. 2018) held state-of-the-art using a single agent in Atari games. In turn, Ape-X achieved state-of-the-art results using multi-agents. R2D2 obtained better results in these environments than the other methods that used a single agent and quadrupled the results obtained by Ape-X. They also pointed out that some methods had not yet obtained above-human performance in many games (52 of the 57) such as R2D2. Still, like the other methods, R2D2 did not show considerable advances in Montezuma's Revenge and Pitfall games, which are known for being difficult environments to explore. The DMLab-30 environment consists of 30 problems in first-person 3D environments and requires long-term memory to obtain reasonable results. According to the authors, while the best-performing algorithms consisted of actor-critic methods trained with some on-policy regime, R2D2 was the first to reach the state-of-the-art using a value-function-based agent.

5.5 Catastrophic forgetting

Rolnick et al. (2019) addressed the problem of catastrophic forgetting that occurs when new experiences overwrite old experiences in the multitasking continuous learning scenario. They proposed a method called Continual Learning with Experience Replay (CLEAR), which sought to balance off-policy learning, using behavioral cloning from experience replay, with on-policy learning, in a trade-off that they define between the concepts of stability (from the preservation of achieved knowledge) and plasticity (from the acquisition of new knowledge). According to the authors, the literature often mitigated catastrophic forgetting by using intensive computational resources to try to learn all tasks simultaneously and not sequentially. However, this problem becomes critical as the application of reinforcement learning in continuous learning problems grows in industry and robotics, with scenarios where rare (and difficult to obtain) experiences may be more common, making simultaneous learning unfeasible. It demands the agent to be able to learn one task at a time in a sequence that is not under the agent's control and whose limits are not unknown. For the authors, this training paradigm eliminates the possibility of simultaneous learning, in which one learns from several tasks, increasing the chance of catastrophic forgetting. However, efforts to prevent catastrophic forgetting have concentrated on approaches that seek to protect the parameters of neural networks inferred in a given task when the agent learns another task, which is motivated by the concepts of consolidation of synapses expected from neuroscience. For Rolnick et al. (2019), many possibilities of using Experience Replay in the scenario of catastrophic forgetting have been ignored in the literature, while the works that used and widely investigated the repetition of experiences did so with a focus on the efficiency of the use of data for agent learning.

To ensure the expected stability from off-policy learning in CLEAR, the authors introduced a method of behavioral cloning between past policies and current policies. Based on ablative studies on three DeepMind Lab tasks, they evaluated the effects of applying their approach to reduce the damage caused by catastrophic forgetting and to verify its behavior in different balances on the trade-off between stability and plasticity, concluding that behavioral cloning is just as crucial to CLEAR performance as using on-policy experiences. They used 900 million frames observed by the agent, varying the size of the replay buffer from 450 million to 5 million experiences. Only in the latter case was some loss of performance observed. The authors also conducted experiments to evaluate the performance of CLEAR varying balances between off-policy and on-policy learning and how these variations impact stability and plasticity, concluding that a trade-off with a percentage of 50/50 led to better results on the DMLab-30 environment, and 75/25 was the best balance on the Atari 2600. Finally, the authors evaluated CLEAR compared to two state-of-the-art methods of reducing catastrophic forgetting that assume task boundaries are known. The authors demonstrated that CLEAR achieved equivalent or better results than both methods despite being simple and agnostic about task boundaries.

According to Rolnick et al. (2019), in those continuous learning cases in which storing experience in a replay buffer is prohibitive, better approaches are in the methods focused on protecting parameters when passing from one task to another. In scenarios where tasks' types and boundaries can be somehow shared, exploiting this can reduce the computational cost or even accelerate agent learning. However, in many cases, such as when the action space changes from one task to another, trying to address past policy distributions, whether

through behavior cloning, off-policy learning, weight protection, or another strategy to prevent catastrophic forgetting, could lead to considerable performance losses. Developing algorithms that selectively forget or protect specific learned behaviors would be necessary in those cases.

5.6 Sparse rewards

Andrychowicz et al. (2017) presented a technique to deal with sparse rewards, one of the significant and challenging problems in Reinforcement learning, because it imposed on the agent to learn long-term policies (until it receives a reward) or to explore an arbitrarily large space of experiences, considering that the immediate rewards are fundamental to guiding the optimal policy's approximation. Despite impacting all methods of RL in different domains, this is a particular issue when dealing with continuous action spaces in robotics-related problems. According to the authors, a common challenge, especially in robotics, is engineering a reward function that reflects the task and can guide policy optimization. Many approaches dedicate efforts to formulating complicated cost functions for problems like stacking a brick on top of another, which limits the application of RL because it requires domain-specific knowledge. Motivated by the way human beings learn almost as much from not-so-good results as from the good or the desired ones, they proposed to bring the reinforcement learning this same idea whenever there are multiple goals to achieve, i.e., achieving each state could be a separate goal. Therefore, their approach was training universal policies, taking input from the current state and a goal state, and replaying each episode with a different goal than the one the agent was trying to achieve, and that was one of the goals achieved in the episode.

After experiencing some episodes, their algorithm called Hindsight Experience Replay (HER) stores every transition resulting from the agent's experiences in the replay buffer, along with the original goal used for this episode and a subset of other goals. As the current goal influences the agent's actions but not the environment dynamics, replaying each trajectory with an arbitrary goal is possible using an off-policy method. According to the authors, one relevant aspect is the strategy they defined to choose the additional goals used for the replay. In the simplest version, they replay each trajectory with the goal achieved in the final state of the episode. However, they have experimentally evaluated different types and quantities of additional goals. In all cases, they also replay each trajectory with the original goal. The authors argue that one can see HER as a form of implicit curriculum because the goals used for replay naturally shift from simple to achieve even by a random agent to more difficult ones. However, HER does not require having any control over the distribution of initial environment states. The experimental results demonstrate that the HER algorithm learns with extremely sparse rewards and performs better with sparse rewards than with shaped ones.

6 Research and applications on ER and some directions for future works

A systematic literature review can show the researcher's interest in Reinforcement Learning using Experience Replay over the last years and point out future directions. We conducted six restriction levels on querying over five indexing databases: CAPES,¹ ACM-DL² (Full-text collection and Guide to Computing Literature), IEEE Explore,³ ScienceDirect,⁴ and Scopus.⁵ From that, we picked up some relevant works that are recent and closely related to the subjects we discussed in this survey. Appendix A presents detailed search results, methodology, and applied criteria. Although we are more interested in works that propose changes or new methods using ER, mainly the ones that investigate it, delving into its theoretical issues and empirical investigations, we could verify that many works in the literature apply well-known RL methods to approach different classes of complex and exciting problems. We present these works in Subsection A and highlight some of them, which, besides focusing on applicated RL, brought interesting approaches in adapting RL and ER methods to make them more suitable to their problems and application domain.

Optimal control is a recurrent problem in the literature that approaches complex non-linear systems, such as robotics, autonomous driving, and domains such as biochemical reactions (Yang et al. 2022). Many research works use variations of policy gradient-based reinforcement learning methods with novel mechanisms of experience replay, such as in Wang et al. (2019), whose proposal was to transform adaptive cruise control problems into optimal tracking control problems and handled by a novel model-free Adaptive Dynamic Programming (ADP) approach called ADPER. Similarly, Yang and He (2020) presented a decentralized Event-triggered Control (ETC) strategy based on Adaptive Critic Learning (ACL) and using experience replay, and Zhou et al. (2022) proposed an attention-based actor-critic algorithm with Prioritized Experience Replay (PER) to improve convergence time on robotic motion planning problems, changing the LSTM-based advantage actor-critic algorithm by using an encoder attention weight and initializing the networks using PER. Kim et al. (2020) introduced a motion planning algorithm for robot manipulators using a twin delayed deep deterministic policy gradient, which applies the Hindsight Experience Replay (HER) formulated by Andrychowicz et al. (2017). Prianto et al. (2020) approached the path planning for multi-arm manipulators by proposing a method based on the Soft Actor-Critic (SAC) algorithm with hindsight experience replay to improve exploration in high-dimensional problems. Sovrano et al. (2022) approached the complex problem of autonomously driving in rule-dense environments by partitioning the experiences buffer into clusters labeled by explanations about rulesets, defining a method called Explanation-Awareness Experience Replay (XAER). Cui et al. (2023) presented their experience replay approach, Double Bias Experience Replay (DBER), and a new loss function (a challenging environment modeling problem in these domains), applied to the classical off-policy algorithms DQN and DDQN and also to the Quantile Regression DQN (QR-DQN). Li and

¹<https://www-periodicos-capes-gov-br.ez1.periodicos.capes.gov.br/index.php>.

²<https://dl.acm.org/>.

³<https://ieeexplore.ieee.org/Xplore/home.jsp>.

⁴<https://www.sciencedirect.com/>.

⁵<https://www.elsevier.com/pt-br/solutions/scopus>.

Ji (2021) proposed a distributed training framework with parallel curriculum experience replay to approach sparse rewards in distributed training in a simulated environment to robots. Hu et al. (2023) describe and evaluate the Asynchronous Curriculum Experience Replay (ACER), which uses multiple threads to update priorities and increases the diversity of experiences. Regarding the memory of experiences, they introduce a temporary pool to improve learning from fresher experiences and change the memory buffer approach from FIFO (First-In, First-Out) to FIOU (First-In, Useless-Out) to enhance learning from old experiences. The authors' main objective was to overcome what they identified as the limitations of PER to seek safe autonomous motion control of unnamed aerial vehicles in complex, unknown environments. Liu et al. (2024) approached the challenges of insufficient data, dynamic uncertainties, long time delay, and slow time-varying of thermal processes in the domain of coordinated control of coal-fired power generation systems. They proposed a DDPG-based method called DPER-VDP3G with a dual-prioritized experience replay and a value distribution strategy to reduce nonuniform sampling bias, remove redundant data, enhance sample diversity, and improve the accuracy of the cost function.

Many works explored novel data structures to make Experience Replay even more data-efficient by modeling the behavior of the transition and exploring predictions about states and rewards (Jiang et al. 2021). Others proposed different forms to define the importance of samples (Kong et al. 2021), new prioritization criteria (Gao et al. 2021), and changes in the updates of the value function approximation to speed up the convergence time by avoiding the incidence of optimal local policy (Kang et al. 2021). In Wei et al. (2022), one can find a new paradigm of replay of experiences (inspired by quantum theory) that considers the complexity and the times replayed of each experience to achieve a better balance between exploration and exploitation, which is a fundamental choice and a complex problem. According to Wang et al. (2024), the imbalanced class distribution may affect the performance of deep reinforcement learning with ER applied to classification problems. Therefore, they approached the problem of customers' credit scoring in P2P lending by modeling it as discrete-time finite Markov decision processes and proposed a balanced stratified prioritized ER strategy to optimize the loss function of a DQN model. Their objective was to balance the numbers of minority and majority experience samples in the mini-batch (according to the class representation) and select more important experience samples for replay based on the principles of PER. The authors defined the concepts and measures of majority and minority experience samples and stored the samples in minority and majority experience replay buffers. They calculated the TD-error for samples from each buffer, calculated the probabilities for prioritization based on the respective TD-errors, and used two value functions and two target functions, one for each sample.

Catastrophic forgetting is another well-known problem affecting training stability and agent performance, especially in continuous action spaces in multi-agent scenarios. It is mainly related to the buffer size limitation and is sensitive to sampling and storage strategies. Recent works addressed this problem by proposing strategies to improve control over the mechanisms for storing, selecting, retaining, and forgetting in experience replay (Osei and Lopez 2023), including using transfer learning about past experiences (Anzaldo and Andrade 2022). According to Li et al. (2021), their Self-generated Long-term Experience Replay (SLER) approach improved the dual experience replay algorithm, applied in continuous learning tasks to mitigate catastrophic forgetting by reducing its impact on computer memory-consuming growth. Li et al. (2022a) proposed a method to cluster and replay expe-

periences using a divide-and-conquer framework based on time division to explore experiences that may not be prioritized during sampling or even forgotten due to limited transition memory.

As discussed in Sect. 4, a (relatively) recent, theoretically relevant, and still little-explored question refers to the trade-off regarding how recent and closer to current policy (and potentially more biased) or how outdated (but more diverse, possibly rare, or expensive to obtain) the experiences in the memory should be to contribute to the agent learning process. Some works approached these questions by changing the priority measure in PER. Ma et al. (2022) proposed to increase the probability of sampling more recent experiences with a novel strategy to replace experiences in the memory buffer, while Zhang et al. (2020) presented a novel self-adaptive priority correction algorithm called Importance-PER to reduce bias. Instead of approaching the sampling strategy, Du et al. (2022) proposed a framework to *refresh* experiences by moving the agent back to past states, executing sequences of actions following its current policy, and storing and reusing new experiences from this process if it turned out better than what the agent previously experienced. Liu et al. (2022) proposed a dynamic experience replay strategy based on Mult-armed Bandit, which combines multiple priority weighted criteria to measure the importance of experiences and adjust their weights from one episode to another. Yang and Peng (2021) introduced the Metalearning-Based Experience Replay (MSER) applied in DDPG to deal with the computational complexity in PER and its need for careful hyperparameter adjustments. They divided the experiences memory buffer into a successful experience buffer and a failure experience buffer and uniformly sampled from those buffers according to a ratio learned by a neural network.

Hindsight Experience Replay (HER) and Aggressive Rewards to Counter Bias in HER (ARCHER) (Lanka and Wu 2018) are two strategies to deal with the classic and challenging problem of sparse rewards but also present some problems. HER considers every failure a success for an alternative (virtual) goal and uses these goals by uniform sampling. However, it introduces bias when not taking variable importance at different training moments and not considering the relevance of goals to agent learning. Vecchiotti et al. (2022) showed that an essential factor in learning multi-goal tasks with the HER is the (relative) rate of hindsight experience used in each training epoch, and it replaces real experience with hindsight experience at a fixed rate during the entire training process. However, their results suggest that hindsight experiences are more relevant at the beginning of training, such as when a robot learns the basic sensing skills and subtasks necessary to achieve the goal. They proposed adjusting the rate of the hindsight experience using a variable sampling rate between the real and hindsight experiences during training. Manela and Biess (2021) proposed to improve HER by prioritizing virtual goals and reducing bias by removing misleading samples. Manela and Biess (2022) presented an algorithm that combines curriculum learning with HER to learn sequential object manipulation tasks for multiple goals and sparse feedback by exploiting the recurrent structure inherent in many object manipulation tasks. Chen et al. (2022a) approached the problem of dealing with sparse rewards in online recommendation systems that use reinforcement learning. They defined a state-aware experience replay model to allow the agent to selectively discover the relevant experiences using locality-sensitive hashing that retains the most meaningful experience at scale and replays more valuable experiences with a higher chance. Dong et al. (2023) proposed the Curiosity-tuned Experience Replay (CTER) method and a curiosity mechanism that generates an intrinsic reward based on a predicted curiosity value to deal with sparse rewards in command

decision modeling for simulated wargaming scenarios. This mechanism also provides an adaptive exploration strategy, a novel prioritized replay, and a more efficient strategy to update the memory of experiences. To improve exploration, they introduced decaying and normalizing factors to guide the agent to explore feasible paths under sparse rewards and a curiosity-adjusted partially greedy exploration to control the ϵ -greedy policy adaptively according to the current level of curiosity of the experience memory. Regarding the ER, they elaborated a curiosity-augmented sampling technique to prioritize experiences by considering both the TD-error and the curiosity. For storing the experiences, they presented a K -segmented curiosity-balanced memory updating approach, which aims to balance the oldness and usefulness of the ER buffer.

Multi-agent and cooperative robot tasks can also take advantage of experience replay and Yu et al. (2023) approached the problem of processing information about the environment while increasing the number of participants by proposing a hybrid attention module integrated with Multi-agent Deep Deterministic policy gradient (HAER-MADDPG) and prioritized experience replay. In turn, Nicholas and Kang (2022) proposed a technique of experience replay that introduces additional strength to the exploration-exploitation trade-off in these scenarios.

Many works in the literature apply well-known RL methods to approach different classes of complex and exciting problems such as: (i) geographical routing-decision process to assign sensing tasks to mobile users; (ii) anomaly detection in smart environments; (iii) cellular-connected unmanned aerial vehicles network; (iv) nonlinearities and uncertainties of biochemical reactions in wastewater treatment process control; (v) robotic lever control; (vi) handover decision in 5 G Ultradense Networks; and (vii) automation of software test (Tao and Hafid 2020; Fährmann et al. 2022; Koroglu and Sen 2022; Crowder et al. 2021; Li et al. 2022b; Wu et al. 2022; Remman and Lekkas 2021; Rosenbauer et al. 2020).

In unmanned aerial vehicles (UAVs), each node can communicate with the others through a routing protocol in UAV ad hoc networks (UANETs). However, UAV routing protocols face the challenges of high mobility and limited node energy, leading to unstable links and sparse network topology due to premature node death. Therefore, Zhang and Qiu (2022) proposed the Deep Reinforcement-Learning-based Geographical Routing Protocol for UANETs to consider link stability and energy prediction called DSEGR. They use the Autoregressive Integrated Moving Average (ARIMA) model to predict the residual energy of neighbor nodes and a link stability evaluation indicator. They modeled the packet forward process as an MDP and used a DDQN with PER to learn the routing decision process. They also designed a reward function to obtain a better convergence rate, and the Analytic Hierarchy Process (AHP) was used to analyze the weights of the factors considered in the reward function. Finally, the authors conducted simulation experiments with DSEGR to analyze network performance, and the results demonstrate that their proposal outperforms others in packet delivery ratio and has a faster convergence rate.

According to Shi et al. (2024), UAVs equipped with mobile edge computing servers have become an emerging technology that provides computing resources for mobile devices to effectively relieve the computational pressure of massive data in 6 G wireless networks. Therefore, they investigated a Multi-UAV Collaborative Assisted Mobile Edge Computing architecture to optimize the computational costs and attempt to reduce consumption of the limited onboard energy, which jointly optimizes the UAV trajectories and the scheduling strategies for the mobile devices offloading. They converted this non-convex optimization

problem with high-dimensional continuous actions into an MDP and proposed the UAVs-assisted Offloading Strategy based on Collaborative Multi-Agent RL (UOS-RL). Due to the highly dynamic variation of the environment, they also presented an experience prioritization mechanism to improve the training efficiency in this scenario. The simulation results demonstrate that the proposed PER-UOS-RL algorithm outperforms existing works relating computational cost.

According to Panda et al. (2024), microgrids are self-supporting generation sources that incorporate renewable energy sources. However, managing the batteries' charge–discharge level is essential while considering the devices' smoother long-term efficiency and reliability. An RL-based strategy can provide instructions for generating pulse width modulation signals in grid-connected inverters. These inverters possess bidirectional power exchange capabilities, enabling them to regulate the direction and magnitude of power flow between the battery and the utility grid, and the agents are trained on real-world sensor readings in practical scenarios to govern the inverter's operations, thereby managing the battery's charging and discharging processes. According to the authors, previous approaches used Deep Q-learning-based methods with PER. Therefore, they justified and proposed investigating the use of Distributional RL with PER in a residential PV-microgrid setup, exploring various algorithms. They focused on energy management to reduce net power imported from the grid, paying particular attention to formulating the penalty function so that the battery does not operate at extreme limits; in addition, a complicated reward function can cause slower convergence in the learning process. They (i) benchmarked different algorithms with PER, (ii) analyzed the deep distributional and Q-learning algorithm training performance with varied discretized action space, random experience replay, and penalty without ToU-induced corrective action, and (iii) analyzed battery operation performance.

The batch process produces relatively few high-value-added products, such as fine chemicals, polymers, and pharmaceuticals. RL is a potential alternative to traditional control methods, such as model predictive control, which seriously affects control performance when the process model is inaccurate. Therefore, Xu et al. (2024) proposed a batch process controller based on Segmented Prioritized Experience Replay (SPER) Soft Actor-Critic (SAC) because it can obtain a more robust control strategy than other RL methods to accurately deal with their complex nonlinear dynamics and unstable operating conditions. SPER is a sampling experience method that the authors designed to improve the efficiency of ER in tasks with long episodes and multiple phases. They also proposed a novel reward function to deal with the sparse reward. They showed the effectiveness of their SPER-SAC-based controller by comparing it with other RL-based control methods.

To address the many problems involving multi-vehicle pursuit, such as autonomous police vehicles that pursue suspects, Li et al. (2024) proposed a multi-agent reinforcement learning algorithm called Progression Cognition Reinforcement Learning with Prioritized Experience for Multi-Vehicle Pursuit (PEPCRL-MVP) in urban multi-intersection dynamic traffic scenes. They used a prioritization network to assess state transitions in the global experience replay buffer according to each agent's parameters, whose mechanisms introduce diversity into the multi-agent learning process and improve collaboration and task-related performance. Furthermore, they employed an attention module to extract critical features from dynamic urban traffic environments and used it to develop a progression cognition method to adaptively group pursuing vehicles. Each group efficiently targeted one

evading vehicle. The authors used a simulator on unstructured roads in an urban area and concluded that PEPCLR-MVP is superior to other state-of-the-art methods.

The recent literature approaches the problems discussed in Sect. 5 in continuous action space, multi-agents, robots and humanoids, and control in complex nonlinear systems such as in the application works presented we presented here. Most proposals are mainly concerned with strategies for sample and data efficiency, and the path the researchers are seeking is clear: speed up and improve the training process in increasingly complex environments and, for that, determine how to explore and exploit the agent's experiences is still a critical question. Moreover, many problems arise from the fundamental issue regarding the limited buffer size, for which simple solutions based on finding an arbitrary or heuristic-measured size seem insufficient, and this aspect needs more attention in the literature. Thinking about a good direction for future work, we suggest investigating the memory of experiences from the perspective of a dynamic-sized structure and looking at the experiences themselves beyond the old, well-known structure that represents a transition in a past time. It is essential to ask how we could work with a more elastic and flexible memory in its structure or how we could explore the relations and dynamics between the agent's experiences and turn this information as relevant as the number of or the priority of the experiences the agent is replaying, in the sense of discussed by Zhang and Sutton (2017) and Neves et al. (2022).

7 Structured summary of literature

This section summarizes the research works, methods, and challenges discussed in this extensive review, whose focus was to contribute to Experience Replay. The main algorithmic strategies and the proposed architectures were the first facets used to subdivide and group the research works, as presented in Fig. 1. All methods address ER in some relevant aspect. The first distinction between research works within the taxonomy is at the individual level, which groups them according to the following criterion. Some works focus on investigating and proposing strategies for specific ER problems related to its formulation and its different methods. These are identified with a green mark in Fig. 1. Other works study and present innovative ways and improvements to existing ER methods applied to complex real-world problems and are identified with a purple mark in Fig. 1. These markings are at the inferior right corner of each box that delimits an article and its authors. The second distinction criterion concerns the strategies of ER each work is approaching, identified by the larger label blocks in the chart vertically, from top to bottom (e.g., naive or non-naive ER? What form of prioritization or relevance does it focus on?). Lastly, the third criterion is the algorithm modifications or architectural changes that each work proposes and uses (e.g., a new neural network architecture, new exploration strategies, or new data structures for the memory of the experience). This criterion can identify more than one distinction for the same work since it may bring contributions in different but combined aspects. This is shown by staking label blocks horizontally from left to right.

The second facet focuses on the research domain, the main problem each work addresses (Table 1), and the proposed approach, organized in Tables 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19. Each table contains the research works corresponding to the larger label blocks in Fig. 1 in chronological order of publication. Tables 2, 3, 4, 5 present the works that employ the original homogenous sampling from a FIFO-like replay buffer

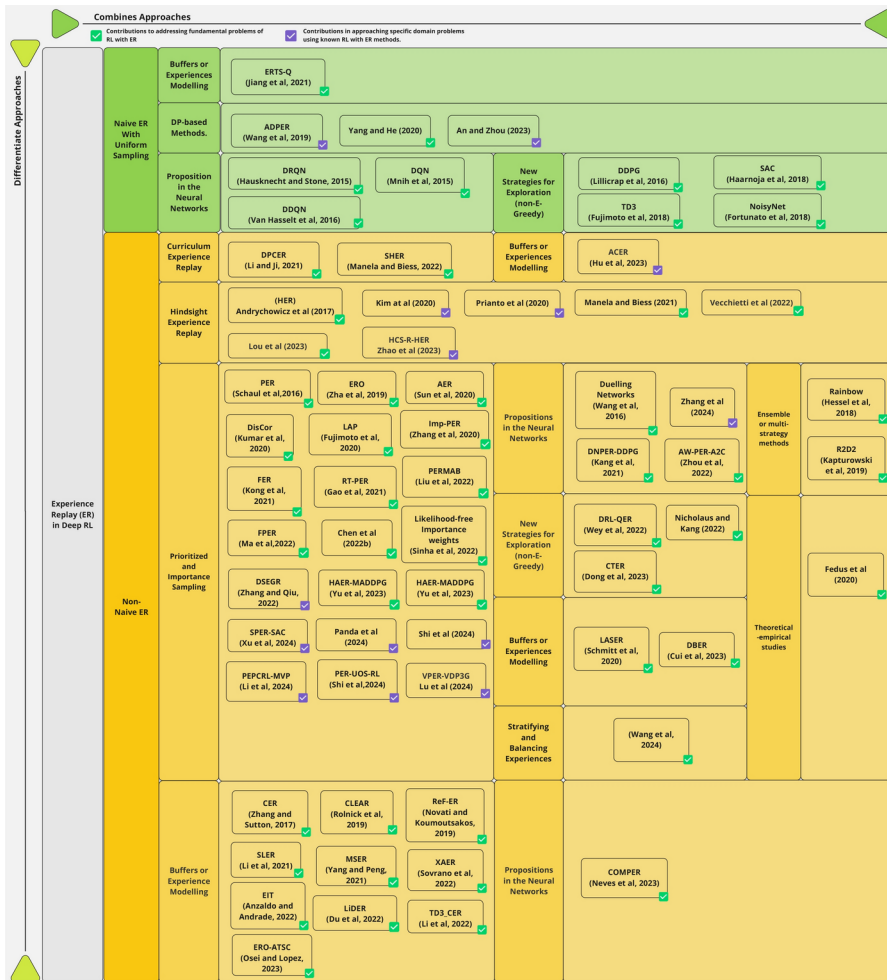


Fig. 1 Diagram of research works organized according to algorithm strategies and proposed architectures. A green mark on the inferior right corner indicates works focusing on strategies for specific ER problems, while a purple mark is used for works focusing on improving existing ER methods applied to complex real-world problems

Table 2 Naive ER with uniform sample plus buffers or experiences modeling

Reference	Domain	Addressing	Approach summary
ERTS-Q (Jiang et al. 2021)	Continuous action space in mountain car and mobile robot	Data efficiency	Defines a tree structure with ER. Establishes a virtual model for subsequent and different states. Aggregates states with highly similar variations.

Table 3 Naive ER with uniform sample in DP-based Methods

References	Domain	Addressing	Approach summary
ADPER (Wang et al. 2019)	Continuous action space for optimal control in nonlinear Systems	Data efficiency	Transforms adaptive cruise control into optimal tracking control in a model-free adaptive dynamic programming approach.
Yang and He (2020)	Optimization in uncertain nonlinear interconnected systems	Sample efficiency Data efficiency	Proposes a Decentralized Event-Triggered Control (ETC) strategy. Obtains optimal ETC laws of auxiliary subsystems. Updates the weight vectors in the critic networks using ER.
An and Zhou (2023)	Control in complex nonlinear systems with a cart-pole and a quadrotor unmanned aerial vehicle	Data efficiency	Proposes a data-driven approach that combines Adaptive Dynamic Programming (ADP) with an actor-critic structure.

Table 4 Naive ER with uniform sample plus propositions on DNN

References	Domain	Addressing	Approach summary
DQN (Mnih et al. 2015)	Discrete action space on Atari 2600	Data efficiency	Approximates the action-value function using a CNN elaborated to video frames features extraction and mapping to a Q-values estimates distribution over the agents' actions in the output layer.
DRQN (Hausknecht and Stone 2015)	Discrete action space on Atari 2600	Data efficiency	Learns long-term policy in sparsed rewards problems. Slightly changes the network from DQN by including a recurrence layer after the CNN layers and before the last dense layer.
DDQN (Van Hasselt et al. 2016)	Discrete action space on Atari 2600	Overestimation bias	Mitigates Q-values overestimation bias. Approximates the action-value functions using CNN like in DQN, although using the Double Q-Learn formulation instead of Q-Learn.

Table 5 Naive ER With Uniform Sample plus Propositions on DNN plus Exploration

References	Domain	Addressing	Approach summary
DDPG (Lillicrap et al. 2016)	Continuous action space on physics tasks	Data efficiency	Presents an actor-critic, model-free and off-policy algorithm based on DPG and using ER to learning policies in a competitive level of planning algorithm.
NoisyNet (Fortunato et al. 2018)	Discrete action space on Atari 2600	Exploration efficiency	Adds learned noise to the network weights to induce stochasticity of the agent's policy and aid efficient exploration.
TED3 (Fujimoto et al. 2018)	Continuous action space on OpenAI Gym	Data efficiency	Modifies DDPG to form the algorithm TD3, which considers the interplay between function approximation errors in both policy and value updates.
SAC (Haarnoja et al. 2018)	Continuous control benchmark tasks	Exploration efficiency	Proposes an off-policy actor-critic algorithm based on the maximum entropy reinforcement learning framework. Entropy is a measure of randomness in the policy. Increasing entropy results in more exploration.

but propose relevant architectural and algorithm improvements on methods strongly based on ER. Tables 6, 7, 8 present works whose propositions mainly focus on Curriculum and Hindsight ER strategies. Tables 9, 10, 11, 12, 13, 14, 15, 16 present the works whose propositions approach some strategy of experiences prioritizing or importance sampling and those

Table 6 Non-naive ER - curriculum experience replay

References	Domain	Addressing	Approach summary
DPCER (Li and Ji 2021)	Continuous-valued action space in distributed RL	Sample efficiency Data efficiency Sparse rewards	Presents a distributed training framework with a parallel curriculum experience replay to identify the difficulty level of subtasks collected in parallel.
SHER (Manela and Biess 2022)	Ball-throwing in authors' handcraft environments	Sample efficiency Data efficiency Sparse rewards	Proposes a Multigoal-based RL that combines curriculum learning with HER. Exploits the recurrent structure inherent in many object manipulation tasks.

Table 7 Non-naive ER - curriculum experience replay plus buffers or experience modeling

Reference	Domain	Addressing	Approach summary
ACER (Hu et al. 2023)	Autonomous motion control of unmanned aerial vehicles	Sample efficiency	Proposes an asynchronous curriculum experience replay to increase the diversity of experiences. Uses multithreads to update and assign priorities.

Table 8 Non-Naive ER - Hindsight Experience Replay

References	Domain	Addressing	Approach summary
SAC-based path planning algorithm (Prianto et al. 2020)	Multi-arm manipulators in high-dimensional problems	Application sample efficiency	Proposes a SAC-based algorithm with hindsight experience replay and configuration space augmentation.
Kim et al. (2020)	Motion and task planning for robot manipulators in manufacturing industry	Application sample efficiency	Defines a Robot Arm Markov Decision Process and approaches it by proposing an algorithm using hindsight and TD3.
Manela and Biess (2021)	Ball-throwing in authors' handcraft environments	Sample efficiency Data efficiency	Proposes the techniques IBS and Filtered HER. Prioritizes virtual goals and reduces bias by removing misleading samples. Exploits the recurrent state-space structure.
HER with sampling rate decay (Vecchietti et al. 2022)	Robot control in multigoal tasks with sparse reward in OpenAI Gym	Exploration efficiency Sample efficiency	Defines a variable sampling rate decay strategy for sampling in Hindsight ER to increase the number of experiences and reduce the original method bias.
RHER and SGES (Luo et al. 2023)	Single-object and multi-object tasks from OpenAI Gym.	Exploration efficiency Aparse rewards	Proposes a self-guided continual RL framework and a self-guided exploration strategy to decompose a sequential task into several subtasks with increasing complexity.
HCS-R-HER (Zhao et al. 2023)	Hierarchical task control of continuous action space in robots motion.	Exploration efficiency Aparse rewards	Proposes a data enhancement algorithm and a two-tier structure containing upper-level and lower-level controllers for decision-making and atomic operations.

Table 9 Non-naive ER - prioritized and importance sampling

References	Domain	Addressing	Approach summary
PER Schaul et al. (2016)	Discrete action space on Atari 2600	Sample efficiency	Prioritizes experiences according to the magnitude of the TD-Error. Combines stochastic sampling priority values and importance sampling strategy.
ERO-ATSC ERO-ATSC+TSC Zha et al. (2019)	Pendulum control in continuous action space	Catastrophic forgetting Data efficiency	Proposes an ER optimization strategy by using dual memory, an alternating transition selection control, and a transition storage control to mitigate catastrophic forgetting.
AER Sun et al. (2020)	Discrete action space on Atari 2600	Data efficiency	Prioritizes transitions containing states more frequently observed by current policy (by similarity) as an heuristic for sampling.
DisCor Kumar et al. (2020)	Continuous and discrete action spaces in multi-task on Atari 2600	Data efficiency Sample efficiency	Proposes modifications in the ADP algorithm to change the buffer's data distribution and reweight the transitions used for training to deal with delayed rewards.
LAP (Fujimoto et al. 2020)	Continuous and discrete action spaces on Mujoco and Atari 2600	Sample efficiency	LAP simplifies PER. Removes unnecessary importance sampling ratios and reduces bias and the likelihood of near-zero-sampling probability experiences. PAL is a LAP- equivalent loss for uniform sampling. It resembles a weighted variant of the Huber loss and computes the same expected gradient as LAP.
Imp-PER (Zhang et al. 2020)	Discrete and continuous action space on Mujoco and Atari 2600	Data efficiency Sample efficiency	Proposes a self-adaptive priority correction algorithm, which predicts the sum of real TD-error of all data in the buffer to define an importance weight. Applies a truncated importance sampling with a self-adaptive truncation threshold.
FER (Kong et al. 2021)	Continuous action space on Mujoco and CARLA	Sample efficiency	Employs a half-normal sampling probability window that allocates a higher sampling probability to newer experiences in the buffer. Proposes a general and local size adjustment schemes that determine the standard deviation of the half-normal sampling window.
RT-PER (Gao et al. 2021)	Continuous action space in Pendulum control on OpenAI Gym	Sample efficiency	Combines the reward value with the TD-error to form an experience prioritization parameter and use it as a standard to update and store the memory pool based on the priority.
PERMAB (Liu et al. 2022)	Discrete action space on a Multi-armed bandit mechanism.	Sample efficiency	Proposes a dynamic ER strategy that can adaptively combine multiple priority criteria to measure the experience's importance. The weight of each assessing criterion can be adaptively adjusted according to their respective contribution to the agent's performance.
FPER (Ma et al. 2022)	Discrete and continuous action spaces on OpenAI Gym	Sample efficiency	Modifies PER based on experience freshness, using a freshness discounted factor as a new hyperparameter and a novel experience replacement strategy in the buffer.
Sinha et al. (2022)	Online recommendation	Sample efficiency Sparse rewards	Proposes a locality-sensitive hashing method to selectively retain the most meaningful experience at scale, together with a reward- driven strategy to prioritize more valuable experiences to deal with sparse rewards.

Table 10 Non-naive ER - prioritized and importance sampling (cont.)

References	Domain	Addressing	Approach summary
Chen et al. (2022b)	Discrete and continuous action space on Atari 2600 and Deep-Mind Control Suite	Sample efficiency	Reweights experiences based on their likelihood under the stationary distribution of the current policy. Uses a likelihood-free density ratio estimator to balance bias from on-policy and variance from off-policy experiences by using the learned ratios as the prioritization weights.
HAER-MADDPG (Yu et al. 2023)	Collaborative Multi-robot in a predator-prey game	Sample efficiency	Uses a hybrid attention-oriented ER in a multi-agent DDPG algorithm, which gives more attention to the states and actions of other robots.
DSEGR (Zhang and Qiu 2022)	Link stability and energy prediction in UANETs	Sample efficiency	Models the packet forward as an MDP and uses a DDQN with PER to learn the routing decision process. Designs a reward function for better convergence.
PER-UOS-RL (Shi et al. 2024)	Computational cost optimization in Mobile Edge Computing servers.	Sample efficiency	Converts the non-convex optimization and high-dimensional continuous action space problem into an MDP. Based on collaborative Multi-Agent RL, proposes a PER mechanism to improve the training efficiency in this scenario.
Panda et al. (2024)	Management of batteries' charge-discharge level in renewable energy sources	Sample efficiency	Investigates and compares the use of different Distributional RL algorithms with PER in a residential PV-microgrid setup. Formulates a non-trivial penalty and reward function and benchmark it based on the battery operations' performance analyses.
SPER-SAC (Xu et al. 2024)	Industry batch process control	Sample efficiency Sparse rewards	Proposes a controller based on Segmented PER (SPER) with SAC to accurately deal with complex nonlinear dynamics and unstable operating conditions, improving the performance of ER in tasks with long episodes and multiple phases. Defines a novel reward function to deal with sparse rewards.
PEPCRL-MVP (Li et al. 2024)	Multi-vehicle Pursuit (MVP) in urban multi-intersection dynamic traffic	Sample efficiency	Proposes a multi-agent RL with PER and a prioritization network to assess experiences according to each agent's parameters. It employs an attention module to extract critical features from the dynamics of the environments and uses this information to develop a progression cognition method to adaptively group pursuing vehicles.
VPER-VDP3G (Liu et al. 2024)	Coordinated control of coal-fired power generation systems.	Sample efficiency Data efficiency	Proposes a dual-prioritized experience replay and a value distribution strategy to reduce nonuniform sampling bias, remove redundant data, enhance sample diversity, and improve the accuracy of the cost function.

works that combine them with some architectural or algorithm propositions (e.g., PER with changes in the neural network and a new method for exploration). Tables 17, 18, 19 present research works that aim to improve ER in sample and data efficiency by directly focusing on architectural and algorithm improvements (e.g., by proposing new data structures for the memory of experiences, compacting or extending the memory, or applying recurrent neural networks).

Table 11 Non-naive ER - prioritized and importance sampling plus proposition on DNNs

References	Domain	Addressing	Approach summary
Dueling network architectures (Wang et al. 2016)	Discrete action space on Atari 2600.	Data efficiency	Uses two streams in the network to approximate the state-value function and a state-dependent action advantage function, whose outputs are the state-value and a vector of advantages for each action. Combining these outputs produces final Q-value.
DNPER-DDPG (Kang et al. 2021)	Continuous action space on OpenAI Gym	Sample efficiency	Proposes a double network PER mechanism in which the minimum estimates of the action-value functions are used to update the actor policy. Uses the Q-values from the two networks and the immediate reward as the criteria for prioritization. Divides the importance of the samples to improve the convergence time.
AW-PER-A2C (Zhou et al. 2022)	Robotic motion planning	Sample efficiency	Represents the features of the robot and the obstacles using an encoder attention weight and combines online and offline A2C with PER.
DDQN-PER (Zhang et al. 2023)	Power consumption planning and collaborative charging control strategy for electric vehicles	Sample efficiency Sparse rewards	Proposes a charging control strategy based on DDQN with PER, which employs an LSTM to address the uncertainties caused by power requirements.

Table 12 Non-naive ER - prioritized and importance sampling plus strategies for exploration

References	Domain	Addressing	Approach summary
DRL-QER (Wei et al. 2021)	Discrete-valued action space on Atari 2600	Exploration efficiency	Adaptively replays experiences according to their complexity and a replaying count to balance exploration and exploitation. Formulates the experiences in quantum representations. An operation of preparation reflects the relationship between the TD-errors and the importance of the experiences. In contrast, an operation of depreciation ensures the transitions' diversity.
Nicholaus and Kang (2022)	Multi-Agent cooperation	Exploration efficiency	Presents a technique that finds suitable experiences in a multi-agent scenario by implicitly introducing additional strength to the exploration-exploitation trade-off. It filters the experiences by computing the similarities between the observed state and previous states in the experiences. This value is used as a hyperparameter to decide which experiences will suit.
CTER (Dong et al. 2023)	Wargaming decision modeling continuous action space on OpenAI Gym	Sample efficiency Sparse rewards Exploration efficiency	Proposes an assembled curiosity-based intrinsic reward that applies decaying and normalizing factors to guide the agent in exploring feasible paths under sparse rewards. Develops a curiosity- adjusted partially greedy exploration to control the ϵ -greedy policy adaptively. Proposes a novel curiosity-augmented sampling technique to prioritize the experience replay and a novel K -segmented curiosity-balanced experience memory updating approach.

Table 13 Non-naive ER - prioritized and importance sampling plus buffers or experiences modeling

References	Domain	Addressing	Approach summary
LASER (Schmitt et al. 2020)	Discrete-valued action space on Atari 2600	Sample efficiency	From the analyses of the bias-variance trade-offs in the form of importance sampling for actor-critic, called V-trace, proposes a new trust region scheme. It mixes experience samples with on-policy experience that scales effectively to data distributions where V-trace becomes unstable.
DBER (Cui et al. 2023)	Visual and sensor data on multi-input autonomous driving	Sample efficiency	Proposes a double-bias experience replay and a new loss function to balance negative and positive loss. Stores nonuniform samples using a biased storage strategy, which learns the weight of samples on a large buffer for positive samples and a smaller buffer for negative samples to separate the types of learning samples. The learning weight of negative samples experience is degraded in the process of loss function calculation.

Table 14 Non-naive ER - prioritized and importance sampling plus stratifying and balancing experiences

Reference	Domain	Addressing	Approach summary
DQN-BSPER (Wang et al. 2024)	Class-imbalanced customer credit score in P2P lending.	Sample efficiency	Proposes a balanced stratified prioritized ER strategy to optimize the loss function of a DQN model. The objective is to balance the numbers of minority and majority experience samples in the mini-batch (according to the class representation) and select more important experience samples based on the principles of PER.

Table 15 Non-naive ER - prioritized and importance sampling in ensemble or multi-strategy methods

References	Domain	Addressing	Approach summary
Rainbow (Hessel et al. 2018)	Discrete action space on Atari 2600	Sample efficiency Data efficiency Exploration efficiency	Adapts PER to use the KL loss of the Distributional Q-Learning. Replaces the one-step distributional loss with a multi-step variant. Defines the target distribution by contracting the value distribution and shifting it by the truncated n-step discounted return. Combines the multi-step distributional loss with Double Q-Learning. Uses the greedy strategy to select and evaluate the action using the online and the target networks. Adapts the dueling architecture for return distributions, uses noisy layers, and factorizes Gaussian to reduce the number of independent noise variables.
R2D2 (Kapturowski et al. 2019)	Discrete action space on Atari 2600	Data efficiency	Investigates RNNs with ER and demonstrates its effect on the parameter lag, which leads to representational drift and recurrent state staleness, mainly in the distributed training setting. Presents an empirical study into the effects of several approaches with RNN, mitigating these effects. Combines RNN to learn representations, DDQN with n-step returns, dueling networks with an additional LSTM layer, Prioritized Distributional ER, and experiences from agents training in parallel.

Table 16 Non-naive ER - prioritized and importance sampling - theoretical-empirical studies

Reference	Domain	Addressing	Approach summary
Fedus et al. (2020)	Discrete action space on Atari 2600	Replay buffer size Sample efficiency	Presents a systematic analysis of ER in Q-learning methods, focusing on replay capacity, replay ratio, and n-step returns. Demonstrates that greater capacity increases the performance of some algorithms but not others and that the theoretically ungrounded, uncorrected n-step returns are uniquely beneficial. In contrast, other techniques confer limited benefits for sifting through larger memory.

Table 17 Non-naive ER - buffers or experience modeling

References	Domain	Addressing	Approach summary
CER Zhang and Sutton (2017)	Discrete action space on Atari 2600	Buffer size Data efficiency	Proposes a new hyperparameter for the buffer size and demonstrates that both a small and a large buffer can heavily hurt the learning process. Remediates the negative influence of a large buffer by adding the current transition to the sampled batch and using the corrected batch to train the agent.
CLEAR (Rolnick et al. 2019)	DeepMind Lab tasks	Catastrophic forgetting	Introduces a method that reduces catastrophic forgetting in multi-task RL by leveraging off-policy learning, behavioral cloning from replay to enhance stability, and on-policy learning to preserve plasticity, which does not require any knowledge of the tasks being learned.
ReF-ER (Novati and Koumoutsakos 2019)	Continuous action space on OpenAI Gym	Data efficiency	Computes the similarity between the current transition and past experiences to skip gradients from those too unlikely with the current policy. It regulates policy changes within a trust region of the replayed behaviors, limits the fraction of far-policy samples in the buffer, and computes gradient estimates only from near-policy experiences.
SLER (Li et al. 2021)	Multiple tasks in sequence.	Catastrophic forgetting	Proposes a short-term ER to retain current task experience and a long-term ER to keep all past experiences to achieve continual learning. A sample model generates the simulated state sequence of the previous tasks for knowledge retention. This is combined with the experience of the new task, generating the simulation of the experience for all previous tasks to alleviate forgetting.
MSER (Yang and Peng 2021)	Trajectory planning of robot manipulator	Sample efficiency	Proposes a meta-learning-based ER buffer separation method that divides the original buffer into a successful experience buffer and a failure experience buffer, with samples randomly from the two buffers according to a ratio learned through a neural network.
XAER (Sovrano et al. 2022)	Discrete and continuous navigation in rule-dense and exception-ridden environments	Sample efficiency	Presents a method for partitioning the buffer into clusters labeled on a per-explanation basis in discrete and continuous navigation environments compatible with modular rulesets. Converts rule-based explanations into case-based explanations and samples experiences in a curricular and task-oriented manner.
EIT (Anzaldo and Andrade 2022)	Power allocation algorithms	Data efficiency Catastrophic forgetting	Proposes a strategy based on a dual memory to mitigate catastrophic forgetting. It exploits previously trained and current data knowledge of the distributional RL model and evaluates the effects of additional training by reusing knowledge from the source model to train new models.

Table 18 Non-naive ER - buffers or experience modeling (cont.)

References	Domain	Addressing	Approach summary
LiDER (Du et al. 2022)	Discrete action space on Atari 2600	Sample efficiency	Refreshes the experiences in the buffer in a multi-agent training by moving the agent back to a past state, executing a sequence of actions by following its current policy, and storing and reusing the new experiences if they are better than previously experienced.
TD3_CER (Li et al. 2022a)	Continuous action space on Mujoco in OpenAI Gym	Exploration efficiency	Exploits the experiences by clustering and replaying them using a divide-and-conquer framework that divides the training process into several periods. It clusters the transitions at the end of each period and defines a conditional probability density function to ensure that all transitions are sufficiently replayed in the current training.
ERO-ATSC (Osei and Lopez 2023)	Continuous action space in OpenAI Gym	Sample efficiency Data efficiency Catastrophic forgetting	Proposes an ER optimization method combining a replay policy network, dual memory, and an alternating two transition selection control mechanism. The first is a hybrid of ER optimization and dual memory, and the second integrates a transition storage control into the first one.

Table 19 Non-naive ER - buffers or experience modeling plus propositions on DNNs

Reference	Domain	Addressing	Approach summary
COMPER (Neves et al. 2022)	Discrete action space in Atari 2600	Replay buffer size Data efficiency	Proposes a method to reduce the convergence time regarding the total number of observations or agent training iterations. Uses TD-Learning with predicted target values for similar transitions sampled from a Compact Transitions Memory, which an LSTN generates by reducing a Transition Memory modeled as sets of similar transitions. This LSTM learns the similar transition sets dynamics regarding its respective observed Q-values to predict the Q-target value at the Q-value function update.

8 Conclusions

This work demonstrates that Experience Replay is a fundamental idea with many open theoretical and empirical problems still being investigated to understand its contributions and propose improvements and new applications with different reinforcement learning methods to solve complex problems in many research fields. Automation, robotics, autonomous driving, trajectory planning, and optimization are among the many application areas that lead to the proposition of new methods of reinforcement learning, as well as new approaches and techniques of experience replay so that these methods can become even more efficient in using data from transitions experienced by agents. New schemas for prioritization of experiences and importance sampling, techniques to avoid catastrophic forgetting, dealing with sparse rewards, and improving memory efficiency in multi-agent environments are among the research explicitly dedicated to improving experience replay.

Appendix A: indexing database query results

To query the indexing databases, we defined six query strings as follows, whose criteria transform the least restrictive search into a fully restrictive one regarding the subject of the research works and its relation with Experience Replay and Reinforcement Learning. We set the range for all databases from the earliest year until the search date. Table 20 presents the count of the results returned by applying these queries to each database after removing duplicates.

- Q1 = Search for (“Experience Replay” AND (“Reinforcement Learning” OR “Deep Reinforcement Learning”)) in all (any) sections.
 - Q2 = Search for (“Experience Replay” AND (“Reinforcement Learning” OR “Deep Reinforcement Learning”)) in the document title, keywords, and abstract.
 - Q3 = Search for (“Reinforcement Learning” OR (“Experience Replay” OR “Deep Reinforcement Learning”)) in the document title AND “Experience Replay” in the keyword OR “Experience Replay” in the abstract.
 - Q4 = Search for (“Reinforcement Learning” OR (“Experience Replay” OR “Deep Reinforcement Learning”)) in the document title AND “Experience Replay” in the keyword AND “Experience Replay” in the abstract.
 - Q5 = Search for “Experience Replay” in the document title AND (“Reinforcement Learning” OR “Deep Reinforcement Learning”) in the keywords AND (“Reinforcement Learning” OR “Deep Reinforcement Learning”) in the abstract.
 - Q6 = Search for “Experience Replay” in the document title AND (“Reinforcement Learning” OR “Deep Reinforcement Learning”) AND “Experience Replay” in the keywords AND (“Reinforcement Learning” OR “Deep Reinforcement Learning” AND “Experience Replay”) in the abstract. The number of publications grew after 2015, presenting numbers much higher than in previous years and continuously rising until 2023. We could observe this behavior at all levels of restrictions on research criteria. We analyze the results for each search to understand their distribution by publication type, year, journal, and publisher. We also identified and grouped the keywords across the whole set and by year of publication to get an initial idea of the problems associated with the main terms of our searches. After this initial analysis, we applied the following criteria, in the order presented, to select up to 40 research papers to read.
1. Only works published after 2018;
 2. Works remaining in each base after applying the query with the most restrictive criteria, starting in Q6;
 3. Works published in high-impact journals;
 4. Works published in high-impact conferences; and
 5. If there are still eligible works and the maximum number has not been reached, return item two and consider the following decreasing restriction query (Q5...Q1).

Table 20 Number of results for each query

Database	Q1	Q2	Q3	Q4	Q5	Q6
ACMDL-full-text	471	170	41	8	5	5
ACMDL-Guide	1577	543	150	31	20	15
CAPES	1000	656	104	61	43	*
IEEE Xplore	427	*	309	84	45	33
ScienceDirect	1099	97	84	*	20	*
Scopus	5128	652	626	231	132	88

In some cases, it was not possible to execute the query because of limitations in the search form (indicated by *)

A1 - ACM-DL

Table 21 shows the results of applying each query in the ACM-DL Full Text Collection and the ACM-DL Guide to Computing Literature. Figures 2 and 3 illustrate the distribution of results from Q1 (the least restrictive query) over the years on the ACM-DL Full-Text Collection and the ACM-DL Guide to Computing Literature, respectively.

Table 21 Distribution of the publication types on the ACM-DL full-text collection and the ACM-DL guide to computing literature

Queries	Results by level of restriction											
	ACM-DL full text						ACM-DL - guide					
	Q1	Q2	Q3	Q4	Q5	Q6	Q1	Q2	Q3	Q4	Q5	Q6
Total	471	170	41	8	5	5	1577	543	149	31	20	15
Inproceeding	346	155	36	7	4	4	668	257	83	14	14	6
Article	106	15	5	1	1	1	848	286	57	17	6	9
Proceedings	16	0	0	0	0	0	16	0	0	0	0	0
Inbook	2	0	0	0	0	0	32	0	1	0	0	0
Book	1	0	0	0	0	0	2	0	1	0	0	0
PhD Thesis	0	0	0	0	0	0	11	0	4	0	0	0

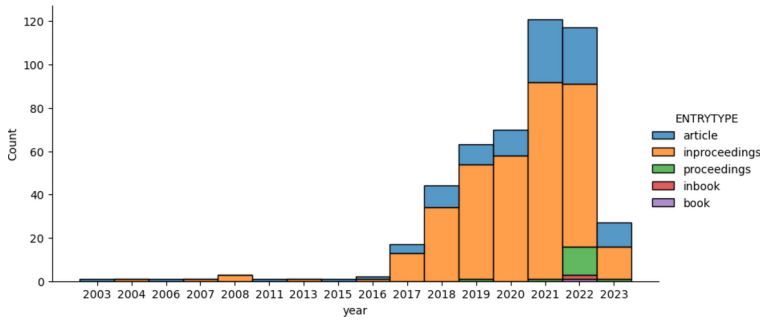


Fig. 2 Q1- Types of publications over the years on the ACM-DL full-text collection

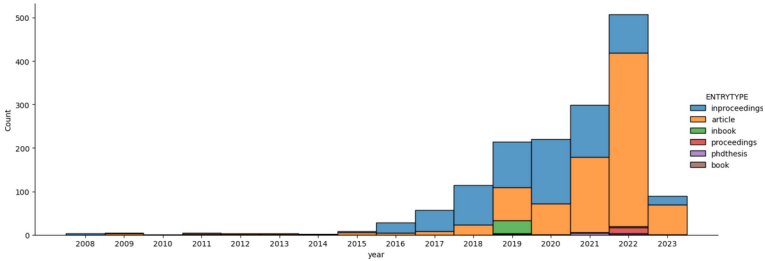


Fig. 3 Q1- Types of publications over the years on the ACM-DL guide to computing literature

A2 - CAPES

Table 22 shows the results of applying each query in CAPES. Figure 4 illustrates the distribution of results from Q1 (the least restrictive query) over the years on CAPES.

Table 22 Distribution of the publication types on CAPES

Queries	Results by level of restriction					
	Q1	Q2	Q3	Q4	Q5	Q6
Total	998	654	104	61	5	*
Inproceedings	0	29	6	5	4	*
Article	998	615	94	54	1	*
Others	0	4	1	1	0	*

In some cases, it was not possible to execute the query because of limitations in the search form (indicated by *)

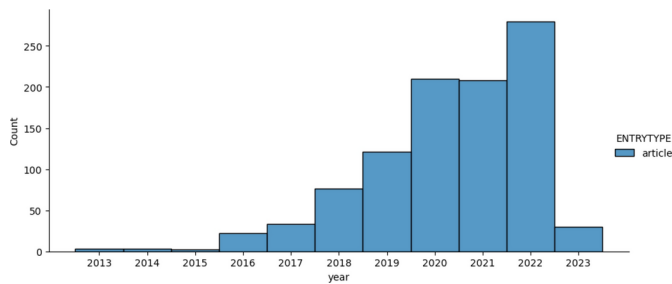


Fig. 4 Q1- Types of publications over the years on CAPES

A3 - IEEE xplora

Table 23 shows the results of applying each query in IEEE Xplore. Figure 5 illustrates the distribution of the results of Q1 (the least restrictive query) over the years on IEEE Xplore.

Table 23 Distribution of the publication types on IEEE Xplore

Queries	Results by level of restriction					
	Q1	Q2	Q3	Q4	Q5	Q6
Total	427	*	309	84	45	33
Inproceedings	256	*	176	48	30	21
Article	171	*	133	36	15	12

In some cases, it was not possible to execute the query because of limitations in the search form (indicated by *)

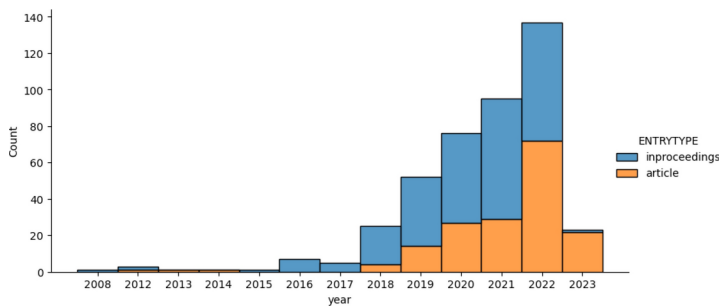


Fig. 5 Q1- Types of publications over the years on IEEE xplora

A4 - Science direct

Table 24 shows the results of applying each query in Science Direct. Figure 6 illustrates the distribution of the results of Q1 (the least restrictive query) over the years on Science Direct.

Table 24 Distribution of the publication types on science direct

Results by level of restriction						
Queries	Q1	Q2	Q3	Q4	Q5	Q6
Total	1099	97	84	*	20	*
Article	1072	0	83	*	20	*
Incollection	27	1	1	*	0	*

In some cases, it was not possible to execute the query due to limitations in the search form (indicated by *)

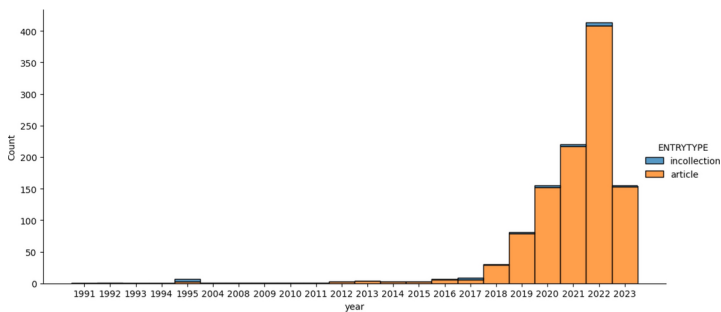


Fig. 6 Q1- Types of publications over the years on direct science

A5 - Scopus

Table 25 shows the results of applying each query in Scopus. Figure 7 illustrates the distribution of results from Q1 (the least restrictive query) over the years in Scopus.

Table 25 Distribution of the publication types on Scopus

Queries	Results by level of restriction					
	Q1	Q2	Q3	Q4	Q5	Q6
Total	5128	851	626	231	132	88
Article	2762	453	342	127	61	45
Conference	2185	363	260	96	66	40
Book	19	1	0	0	0	0

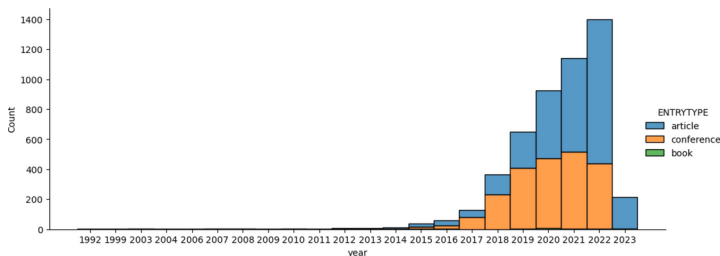


Fig. 7 Q1- Types of publications over the years on scopus

Acknowledgements This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001. It has also received partial funding from Brazilian Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) and Pontifical Catholic University of Minas Gerais (PUC Minas).

Author contributions D. E. N. wrote the main manuscript text. Z. K. G. P. J. and L.I. supervised the writing and reviewed the manuscript.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- An C, Zhou J (2023) Adaptive dynamic programming for data-based optimal state regulation with experience replay. *Neurocomputing* 554(126):616. <https://doi.org/10.1016/j.neucom.2023.126616> (<https://www.sciencedirect.com/science/article/pii/S092523231223007397>)
- Andrychowicz M, Wolski F, Ray A, et al. (2017) Hindsight experience replay. In: Guyon I, Luxburg UV, Bengio S, et al. (eds) *Advances in neural information processing systems (NIPS 2017)*
- Anzaldo A, Andrade AG (2022) Experience replay-based power control for sum-rate maximization in multi-cell networks. *IEEE Wirel Commun Lett* 11(11):2350–2354. <https://doi.org/10.1109/LWC.2022.3202904>
- Bellemare MG, Naddaf Y, Veness J et al. (2013) The arcade learning environment: an evaluation platform for general agents. *J Artif Intell Res* 47:253–279. <https://doi.org/10.1613/jair.3912>
- Bellemare MG, Dabney W, Munos R (2017) A distributional perspective on reinforcement learning. In: *International conference on machine learning*, PMLR, pp 449–458
- Brockman G, Cheung V, Pettersson L, et al. (2016) Openai gym. *arXiv preprint arXiv:1606.01540*<https://doi.org/10.48550/arXiv.1606.01540>
- Castro PS, Moitra S, Gelada C, et al. (2018) Dopamine: a research framework for deep reinforcement learning. *arXiv preprint* <https://doi.org/10.48550/arXiv.1812.06110>
- Chen X, Yao L, McAuley J, et al. (2022a) Locality-sensitive state-guided experience replay optimization for sparse rewards in online recommendation. In: *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*, pp 1316–1325, <https://doi.org/10.1145/3477495.3532015>
- Chen X, Yao L, McAuley J, et al. (2022b) Locality-sensitive state-guided experience replay optimization for sparse rewards in online recommendation. In: *SIGIR 2022 - Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. Association for Computing Machinery, Inc, pp 1316–1325, <https://doi.org/10.1145/3477495.3532015>
- Crowder DC, Abreu J, Kirsch RF (2021) Hindsight experience replay improves reinforcement learning for control of a mimo musculoskeletal model of the human arm. *IEEE Trans Neural Syst Rehabil Eng* 29:1016–1025. <https://doi.org/10.1109/TNSRE.2021.3081056>
- Cui J, Yuan L, He L et al. (2023) Multi-input autonomous driving based on deep reinforcement learning with double bias experience replay. *IEEE Sens J* 23(11):11253–11261. <https://doi.org/10.1109/JSEN.2023.3237206>
- Dabney W, Ostrovski G, Silver D, et al. (2018) Implicit quantile networks for distributional reinforcement learning. In: *Proceedings of the 35th international conference on machine learning*, PMLR, pp 1096–1105
- Degrís T, White M, Sutton RS (2012) Off-policy actor-critic. In: *29th International conference on machine learning*
- Dong L, Li N, Gong G (2023) Curiosity-tuned experience replay for wargaming decision modeling without reward-engineering. *Simul Model Pract Theory* 129(102):842. <https://doi.org/10.1016/j.simpat.2023.102842> (<https://www.sciencedirect.com/science/article/pii/S1569190X23001193>)
- Du Y, Warnell G, Gebremedhin A et al. (2022) Lucid dreaming for experience replay: refreshing past states with the current policy. *Neural Comput Appl* 34(3):1687–1712. <https://doi.org/10.1007/s00521-021-06104-5>
- Elharrouss O, Akbari Y, Almadeed N et al. (2024) Backbones-review: feature extractor networks for deep learning and deep reinforcement learning approaches in computer vision. *Comput Sci Rev* 53(100):645. <https://doi.org/10.1016/j.cosrev.2024.100645> (<https://www.sciencedirect.com/science/article/pii/S1574013724000297>)
- Espeholt L, Soyer H, Munos R, et al. (2018) Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. In: *35th International conference on machine learning*, PMLR, pp 1407–1416
- Fährmann D, Jorek N, Damer N et al. (2022) Double deep q-learning with prioritized experience replay for anomaly detection in smart environments. *IEEE Access* 10:60836–60848. <https://doi.org/10.1109/ACCESS.2022.3179720>
- Fedus W, Ramachandran P, Agarwal R, et al. (2020) Revisiting fundamentals of experience replay. In: *International conference on machine learning*, PMLR, pp 3061–3071
- Fortunato M, Azar MG, Piot B, et al. (2018) Noisy networks for exploration. In: *6th International conference on learning representations*
- Fujimoto S, van Hoof H, Meger D (2018) Addressing function approximation error in actor-critic methods. In: Dy J, Krause A (eds) *Proceedings of the 35th international conference on machine learning*, vol 80. PMLR, pp 1587–1596

- Fujimoto S, Meger D, Precup D (2020) An equivalence between loss functions and non-uniform sampling in experience replay. In: Larochelle H, Ranzato M, Hadsell R, et al. (eds) *Advances in neural information processing systems*, vol 33. Curran Associates, Inc., pp 14,219–14,230
- Gao J, Li X, Liu W, et al. (2021) Prioritized experience replay method based on experience reward. In: 2021 International conference on machine learning and intelligent systems engineering (MLISE), IEEE, pp 214–219, <https://doi.org/10.1109/MLISE54096.2021.00045>
- Gu S, Lillicrap T, Sutskever I, et al. (2016) Continuous deep q-learning with model-based acceleration. In: *Proceedings of The 33rd international conference on machine learning*, PMLR, pp 2829–2838
- Haarnoja T, Zhou A, Abbeel P, et al. (2018) Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: *Proceedings of the 35th international conference on machine learning*, PMLR, pp 1861–1870
- van Hasselt H (2010) Double Q-learning. In: *Advances in neural information processing systems 23 (NIPS 2010)*, pp 2613–2621
- van Hasselt HP, Hessel M, Aslanides J (2019) When to use parametric models in reinforcement learning? In: Wallach H, Larochelle H, Beygelzimer A et al. (eds) *Adv Neural Inform Process Syst*, vol 32. Curran Associates Inc
- Hausknecht M, Stone P (2015) Deep recurrent q-learning for partially observable mdps. In: *AAAI fall symposium on sequential decision making for intelligent agents (AAAI-SDMIA15)*
- Hessel M, Modayil J, Van Hasselt H, et al. (2018) Rainbow: combining improvements in deep reinforcement learning. In: *Proceedings of the AAAI conference on artificial intelligence*, <https://doi.org/10.1609/aaai.v32i1.11796>
- Hickling T, Zenati A, Aouf N et al. (2023) Explainability in deep reinforcement learning: a review into current methods and applications. *ACM Comput Surv*. <https://doi.org/10.1145/3623377>
- Horgan D, Quan J, Budden D, et al. (2018) Distributed prioritized experience replay. In: *International conference on learning representations*
- Hu Z, Gao X, Wan K et al. (2023) Asynchronous curriculum experience replay: a deep reinforcement learning approach for uav autonomous motion control in unknown dynamic environments. *IEEE Trans Vehic Technol* 72(11):13985–14001. <https://doi.org/10.1109/TVT.2023.3285595>
- Ioffe S, Szegedy C (2015) Batch normalization: accelerating deep network training by reducing internal covariate shift. In: Bach F, Blei D (eds) *Proceedings of the 32nd international conference on machine learning*, vol 37. PMLR, pp 448–456
- Jiang WC, Hwang KS, Lin JL (2021) An experience replay method based on tree structure for reinforcement learning. *IEEE Trans Emerg Top Comput* 9(2):972–982. <https://doi.org/10.1109/TETC.2018.2890682>
- Łukasz Kaiser, Babaeizadeh M, Milos P, et al. (2020) Model-based reinforcement learning for atari. In: *International conference on learning representations*
- Kaiser L, Babaeizadeh M, Milos P, et al. (2019) Model-based reinforcement learning for atari. *arXiv preprint* <https://doi.org/10.48550/arXiv.1903.00374>
- Kang C, Rong C, Ren W et al. (2021) Deep deterministic policy gradient based on double network prioritized experience replay. *IEEE Access* 9:60296–60308. <https://doi.org/10.1109/ACCESS.2021.3074535>
- Kapturowski S, Ostrovski G, Quan J, et al. (2019) Recurrent experience replay in distributed reinforcement learning. In: *International conference on learning representations*
- Kim M, Han DK, Park JH et al. (2020) Motion planning of robot manipulators for a smoother path using a twin delayed deep deterministic policy gradient with hindsight experience replay. *Appl Sci* 10(2):575. <https://doi.org/10.3390/app10020575>
- Kingma DP, Ba J (2015) Adam: a method for stochastic optimization. In: Bengio Y, LeCun Y (eds) *3rd International conference on learning representations*
- Kong SH, Nahrendra IMA, Paek DH (2021) Enhanced off-policy reinforcement learning with focused experience replay. *IEEE Access* 9:93152–93164. <https://doi.org/10.1109/ACCESS.2021.3085142>
- Koroglu Y, Sen A (2022) Fast witness generation for readable gui test scenarios via generalized experience replay. *IEEE Access* 10:116224–116240. <https://doi.org/10.1109/ACCESS.2022.3218902>
- Kumar A, Gupta A, Levine S (2020) Disco: Corrective feedback in reinforcement learning via distribution correction. In: Larochelle H, Ranzato M, Hadsell R, et al. (eds) *Advances in neural information processing systems*, vol 33. Curran Associates, Inc., pp 18,560–18,572
- Lanka S, Wu T (2018) Archer: aggressive rewards to counter bias in hindsight experience replay. *arXiv:1809.02070*
- Li C, Li Y, Zhao Y et al. (2021) Sler: Self-generated long-term experience replay for continual reinforcement learning. *Appl Intell* 51(1):185–201. <https://doi.org/10.1007/s10489-020-01786-1>
- Li M, Huang T, Zhu W (2022) Clustering experience replay for the effective exploitation in reinforcement learning. *Pattern Recogn*. <https://doi.org/10.1016/j.patcog.2022.108875>
- Li X, Yang Y, Yuan Z et al. (2024) Progression cognition reinforcement learning with prioritized experience for multi-vehicle pursuit. *IEEE Trans Intell Transp Syst*. <https://doi.org/10.1109/TITS.2024.3354196>

- Li Y, Ji J (2021) Parallel curriculum experience replay in distributed reinforcement learning. In: Proceedings of the 20th international conference on autonomous agents and multiagent systems. International Foundation for Autonomous Agents and Multiagent Systems, pp 782–789
- Li Y, Aghvami AH, Dong D (2022) Path planning for cellular-connected UAV: a DRL solution with quantum-inspired experience replay. *IEEE Trans Wirel Commun* 21(10):7897–7912. <https://doi.org/10.1109/TWC.2022.3162749>
- Lillicrap TP, Hunt JJ, Pritzel A, et al. (2016) Continuous control with deep reinforcement learning. In: International conference on representation learning
- Lin LJ (1992) Self-improving reactive agents based on reinforcement learning, planning, and teaching. *Mach Learn* 8(3–4):293–321. <https://doi.org/10.1007/BF00992699>
- Liu R, Zou J (2018) The effects of memory replay in reinforcement learning. In: 2018 56th Annual Allerton conference on communication, control, and computing, pp 478–485. <https://doi.org/10.1109/ALLERTON.2018.8636075>
- Liu X, Zhu T, Jiang C et al. (2022) Prioritized experience replay based on multi-armed bandit. *Expert Syst Appl* 189(116):023. <https://doi.org/10.1016/j.eswa.2021.116023>
- Liu X, Yu M, Yang C et al. (2024) Value distribution ddpg with dual-prioritized experience replay for coordinated control of coal-fired power generation systems. *IEEE Trans Ind Inf* 20(6):8181–8194. <https://doi.org/10.1109/TII.2024.3369712>
- Luo Y, Wang Y, Dong K et al. (2023) Relay hindsight experience replay: self-guided continual reinforcement learning for sequential object manipulation tasks with sparse rewards. *Neurocomputing* 557(126):620. <https://doi.org/10.1016/j.neucom.2023.126620> (<https://www.sciencedirect.com/science/article/pii/S0925231223007439>)
- Ma J, Ning D, Zhang C et al. (2022) Fresher experience plays a more important role in prioritized experience replay. *Appl Sci* 12(23):12489. <https://doi.org/10.3390/app122312489>
- Machado MC, Bellemare MG, Talvitie E et al. (2018) Revisiting the arcade learning environment: evaluation protocols and open problems for general agents. *J Artif Intell Res* 61:523–562. <https://doi.org/10.1613/jair.5699>
- Manela B, Biess A (2021) Bias-reduced hindsight experience replay with virtual goal prioritization. *Neurocomputing* 451:305–315. <https://doi.org/10.1016/j.neucom.2021.02.090>
- Manela B, Biess A (2022) Curriculum learning with hindsight experience replay for sequential object manipulation tasks. *Neural Netw* 145:260–270. <https://doi.org/10.1016/j.neunet.2021.10.011>
- Mckenzie MC, McDonnell MD (2022) Modern value based reinforcement learning: a chronological review. *IEEE Access* 10:134704–134725. <https://doi.org/10.1109/ACCESS.2022.3228647>
- Mishra S, Arora A (2024) Intelligent computational techniques for physical object properties discovery, detection, and prediction: a comprehensive survey. *Comput Sci Rev* 51(100):609. <https://doi.org/10.1016/j.cosrev.2023.100609> (<https://www.sciencedirect.com/science/article/pii/S157401372300076X>)
- Mnih V, Kavukcuoglu K, Silver D, et al. (2013) Playing atari with deep reinforcement learning. In: Deep learning workshop NIPS 2013
- Mnih V, Kavukcuoglu K, Silver D et al. (2015) Human-level control through deep reinforcement learning. *Nature* 518(7540):529–533. <https://doi.org/10.1038/nature14236>
- Mnih V, Badia AP, Mirza M, et al. (2016) Asynchronous methods for deep reinforcement learning. In: Balcan MF, Weinberger KQ (eds) Proceedings of The 33rd international conference on machine learning, Proceedings of machine learning research, vol 48. PMLR, pp 1928–1937
- Moreno-Vera F (2019) Performing deep recurrent double q-learning for atari games. In: 2019 IEEE Latin American conference on computational intelligence (LA-CCI), IEEE, pp 1–4
- Neves DE, Ishitani L, do Patrocínio Júnior ZKG (2022) When less may be more: exploring similarity to improve experience replay. In: Intelligent systems: 11th Brazilian conference, BRACIS 2022, Part II, Springer, pp 96–110. https://doi.org/10.1007/978-3-031-21689-3_8
- Nicholaus IT, Kang DK (2022) Robust experience replay sampling for multi-agent reinforcement learning. *Pattern Recogn Lett* 155:135–142. <https://doi.org/10.1016/j.patrec.2021.11.006>
- Novati G, Koumoutsakos P (2019) Remember and forget for experience replay. In: Chaudhuri K, Salakhutdinov R (eds) Proceedings of the 36th international conference on machine learning, vol 97. PMLR, pp 4851–4860
- Osei RS, Lopez D (2023) Experience replay optimisation via ATSC and TSC for performance stability in Deep RL. *Appl Sci* 13(4):2034. <https://doi.org/10.3390/app13042034>
- Panda DK, Turner O, Das S et al. (2024) Prioritized experience replay based deep distributional reinforcement learning for battery operation in microgrids. *J Clean Prod* 434(139):947. <https://doi.org/10.1016/j.jclepro.2023.139947> (<https://www.sciencedirect.com/science/article/pii/S0959652623041057>)
- Peng P, Lin W, Wu W et al. (2024) A survey on computation offloading in edge systems: from the perspective of deep reinforcement learning approaches. *Comput Sci Rev* 53(100):656. <https://doi.org/10.1016/j.cosrev.2024.100656> (<https://www.sciencedirect.com/science/article/pii/S1574013724000406>)

- Prianto E, Kim M, Park JH et al. (2020) Path planning for multi-arm manipulators using deep reinforcement learning: soft actor-critic with hindsight experience replay. *Sensors* 20(20):5911. <https://doi.org/10.3390/s20205911>
- Remman SB, Lekkas AM (2021) Robotic lever manipulation using hindsight experience replay and shapley additive explanations. In: 2021 European control conference (ECC), IEEE, pp 586–593, <https://doi.org/10.23919/ECC54610.2021.9654850>
- Rolnick D, Ahuja A, Schwarz J, et al. (2019) Experience replay for continual learning. *Adv Neural Inf Process Syst* 32
- Rosenbauer L, Stein A, Pätzel D, et al. (2020) XCSF with experience replay for automatic test case prioritization. In: 2020 IEEE symposium series on computational intelligence (SSCI), IEEE, pp 1307–1314, <https://doi.org/10.1109/SSCI47803.2020.9308379>
- Schaul T, Quan J, Antonoglou I, et al. (2016) Prioritized experience replay. In: Proceedings of the international conference on representation learning
- Schmitt S, Hessel M, Simonyan K (2020) Off-policy actor-critic with shared experience replay. In: III HD, Singh A (eds) Proceedings of the 37th international conference on machine learning, proceedings of machine learning research, vol 119. PMLR, pp 8545–8554
- Shen Y, Zhao X (2024) Reinforcement learning in natural language processing: a survey. In: Proceedings of the 2023 6th international conference on machine learning and natural language processing. Association for Computing Machinery, New York, NY, USA, MLNLP '23, p 84–90, <https://doi.org/10.1145/3639479.3639496>
- Shi H, Tian Y, Li H et al. (2024) Task offloading and trajectory scheduling for uav-enabled mec networks: an madrl algorithm with prioritized experience replay. *Ad Hoc Netw* 154(103):371. <https://doi.org/10.1016/j.adhoc.2023.103371> (<https://www.sciencedirect.com/science/article/pii/S1570870523002913>)
- Silver D, Lever G, Heess N, et al. (2014) Deterministic policy gradient algorithms. In: Xing EP, Jebara T (eds) Proceedings of the 31st international conference on machine learning, vol 32. PMLR, Beijing, China, pp 387–395
- Sinha S, Song J, Garg A, et al. (2022) Experience replay with likelihood-free importance weights. In: Firoozi R, Mehr N, Yel E, et al. (eds) Proceedings of The 4th annual learning for dynamics and control conference, vol 168. PMLR, pp 110–123
- Sovrano F, Raymond A, Prorok A (2022) Explanation-aware experience replay in rule-dense environments. *IEEE Robot Automat Lett* 7(2):898–905. <https://doi.org/10.1109/LRA.2021.3135927>
- Sun P, Zhou W, Li H (2020) Attentive experience replay. *Proc AAAI Conf Artif Intell* 34(04):5900–5907. <https://doi.org/10.1609/aaai.v34i04.6049>
- Sutton RS (1992) Reinforcement learning architectures. In: Proceedings ISKIT'92 international symposium on neural information processing
- Sutton RS, Barto AG (2018) Reinforcement learning: an introduction, 2nd edn. The MIT Press, Cambridge
- Szepesvári C (2010) Algorithms for reinforcement learning, vol 4. Morgan & Claypool Publishers. <https://doi.org/10.2200/S00268ED1V01Y201005AIM009>
- Tao X, Hafid AS (2020) Deepensing: a novel mobile crowdsensing framework with double deep q-network and prioritized experience replay. *IEEE Internet Things J* 7(12):11547–11558. <https://doi.org/10.1109/JIOT.2020.3022611>
- Tassa Y, Doron Y, Muldal A, et al. (2018) Deepmind control suite. arXiv preprint [arXiv:1801.00690](https://arxiv.org/abs/1801.00690) abs/1801.00690. <https://doi.org/10.48550/arXiv.1801.00690>
- Todorov E, Erez T, Tassa Y (2012) Mujoco: a physics engine for model-based control. In: 2012 IEEE/RSJ international conference on intelligent robots and systems, pp 5026–5033, <https://doi.org/10.1109/IROS.2012.6386109>
- Uhlenbeck GE, Ornstein LS (1930) On the theory of the brownian motion. *Phys Rev* 36:823–841. <https://doi.org/10.1103/PhysRev.36.823>
- Van Hasselt H, Guez A, Silver D (2016) Deep reinforcement learning with double q-learning. In: Proceedings of the AAAI conference on artificial intelligence, <https://doi.org/10.1609/aaai.v30i1.10295>
- Vecchietti LF, Seo M, Har D (2022) Sampling rate decay in hindsight experience replay for robot control. *IEEE Trans Cybern* 52(3):1515–1526. <https://doi.org/10.1109/TCYB.2020.2990722>
- Wang B, Zhao D, Cheng J (2019) Adaptive cruise control via adaptive dynamic programming with experience replay. *Soft Comput* 23(12):4131–4144. <https://doi.org/10.1007/s00500-018-3063-7>
- Wang C, Ross KW (2019) Boosting soft actor-critic: Emphasizing recent experience without forgetting the past. arXiv preprint [arXiv:1906.04009](https://arxiv.org/abs/1906.04009) abs/1906.04009. <https://doi.org/10.48550/arXiv.1906.04009>
- Wang Y, Jia Y, Fan S et al. (2024) Deep reinforcement learning based on balanced stratified prioritized experience replay for customer credit scoring in peer-to-peer lending. *Artif Intell Rev* 57(4):93. <https://doi.org/10.1007/s10462-023-10697-9> (<https://doi.org/10.1007/s10462-023-10697-9>)

- Wang Z, Schaul T, Hessel M, et al. (2016) Dueling network architectures for deep reinforcement learning. In: Balcan MF, Weinberger KQ (eds) Proceedings of The 33rd international conference on machine learning, vol 48. PMLR, pp 1995–2003
- Watkins CJCH, Dayan P (1992) Q-learning. *Mach Learn* 8(3):279–292. <https://doi.org/10.1007/BF00992698>
- Wei Q, Ma H, Chen C et al. (2021) Deep reinforcement learning with quantum-inspired experience replay. *IEEE Trans Cybern*. <https://doi.org/10.1109/TCYB.2021.3053414>
- Wei Q, Ma H, Chen C et al. (2022) Deep reinforcement learning with quantum-inspired experience replay. *IEEE Trans Cybern* 52(9):9326–9338. <https://doi.org/10.1109/TCYB.2021.3053414>
- Werbos P (1990) Backpropagation through time: what it does and how to do it. *Proc IEEE* 78(10):1550–1560. <https://doi.org/10.1109/5.58337>
- Wu DF, Huang C, Yin Y et al. (2022) State aware-based prioritized experience replay for handover decision in 5g ultradense networks. *Wirel Commun Mob Comput*. <https://doi.org/10.1155/2022/5006770>
- Xu C, Ma J, Tao H (2024) Batch process control based on reinforcement learning with segmented prioritized experience replay. *Meas Sci Technol* 35(5):056202, <https://doi.org/10.1088/1361-6501/ad21cf>
- Yang J, Peng G (2021) DDPG with meta-learning-based experience replay separation for robot trajectory planning. In: 2021 7th international conference on control, automation and robotics (ICCAR), pp 46–51, <https://doi.org/10.1109/ICCAR52225.2021.9463493>
- Yang R, Wang D, Qiao J (2022) Policy gradient adaptive critic design with dynamic prioritized experience replay for wastewater treatment process control. *IEEE Trans Ind Inf* 18(5):3150–3158. <https://doi.org/10.1109/TII.2021.3106402>
- Yang X, He H (2020) Adaptive critic learning and experience replay for decentralized event-triggered control of nonlinear interconnected systems. *IEEE Trans Syst, Man, Cybern: Syst* 50(11):4043–4055. <https://doi.org/10.1109/TSMC.2019.2898370>
- Yarats D, Kostrikov I, Fergus R (2021) Image augmentation is all you need: regularizing deep reinforcement learning from pixels. In: International conference on learning representations
- Yu L, Huo S, Wang Z et al. (2023) Hybrid attention-oriented experience replay for deep reinforcement learning and its application to a multi-robot cooperative hunting problem. *Neurocomputing* 523:44–57. <https://doi.org/10.1016/j.neucom.2022.12.020>
- Zha D, Lai KH, Zhou K, et al. (2019) Experience replay optimization. In: Proceedings of the twenty-eighth international joint conference on artificial intelligence, IJCAI-19. International joint conferences on artificial intelligence organization, pp 4243–4249, <https://doi.org/10.24963/ijcai.2019/589>
- Zhang H, Qu C, Zhang J et al. (2020) Self-adaptive priority correction for prioritized experience replay. *Appl Sci* 10(19):6925. <https://doi.org/10.3390/app10196925>
- Zhang S, Sutton RS (2017) A deeper look at experience replay. In: 31st conference on neural information processing systems (NIPS 2017)
- Zhang Y, Qiu H (2022) DDQN with prioritized experience replay-based optimized geographical routing protocol of considering link stability and energy prediction for uanet. *Sensors* 22(13):5020. <https://doi.org/10.3390/s22135020>
- Zhang Y, Rao X, Liu C et al. (2023) A cooperative ev charging scheduling strategy based on double deep q-network and prioritized experience replay. *Eng Appl Artif Intell* 118(105):642. <https://doi.org/10.1016/j.engappai.2022.105642> (<https://www.sciencedirect.com/science/article/pii/S0952197622006327>)
- Zhao X, Du J, Wang Z (2023) Hcs-r-her: hierarchical reinforcement learning based on cross subtasks rainbow hindsight experience replay. *J Comput Sci* 72(102):113. <https://doi.org/10.1016/j.jocs.2023.102113> (<https://www.sciencedirect.com/science/article/pii/S1877750323001734>)
- Zhou C, Huang B, Hassan H et al. (2022) Attention-based advantage actor-critic algorithm with prioritized experience replay for complex 2-d robotic motion planning. *J Intell Manuf* 34(1):151–180. <https://doi.org/10.1007/s10845-022-01988-z>
- Zhu C, Dastani M, Wang S (2024) A survey of multi-agent deep reinforcement learning with communication. In: Proceedings of the 23rd international conference on autonomous agents and multiagent systems. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, AAMAS '24, p 2845–2847